

Question 1

You are creating a Cloud Formation template to deploy your CMS application running on an EC2 instance within your AWS account. Since the application will be deployed across multiple regions, you need to create a map of all the possible values for the base AMI. How will you invoke the !FindInMap function to fulfill this use case?

- !FindInMap [MapName]
- !FindInMap [MapName, TopLevelKey, SecondLevelKey, ThirdLevelKey]
- !FindInMap [MapName, TopLevelKey]
- !FindInMap [MapName, TopLevelKey, SecondLevelKey]

Overall explanation

Correct option:

!FindInMap [MapName, TopLevelKey, SecondLevelKey] - The intrinsic function Fn::FindInMap returns the value corresponding to keys in a two-level map that is declared in the Mappings section. YAML

Syntax for the full function name: Fn::FindInMap: [MapName, TopLevelKey, SecondLevelKey]

Short form of the above syntax is : !FindInMap [MapName, TopLevelKey, SecondLevelKey]

Where,
MapName - Is the logical name of a mapping declared in the Mappings section that contains the keys and values. TopLevelKey - The top-level key name. Its value is a list of key-value pairs.
SecondLevelKey - The second-level key name, which is set to one of the keys from the list assigned to TopLevelKey.

Consider the following YAML template:

Mappings:

RegionMap:

```
us-east-1:
  HVM64: "ami-0ff8a91507f77f867"
  HVMG2: "ami-0a584ac55a7631c0c"
us-west-1:
  HVM64: "ami-0bdb828fd58c52235"
  HVMG2: "ami-066ee5fd4a9ef77f1"
eu-west-1:
  HVM64: "ami-047bb4163c506cd98"
  HVMG2: "ami-31c2f645"
ap-southeast-1:
  HVM64: "ami-08569b978cc4dfa10"
  HVMG2: "ami-0be9df32ae9f92309"
ap-northeast-1:
  HVM64: "ami-06cd52961ce9f0d85"
  HVMG2: "ami-053cdd503598e4a9d"
```

Resources:

myEC2Instance:

Type: "AWS::EC2::Instance"

Properties:

```
ImageId: !FindInMap
- RegionMap
- !Ref 'AWS::Region'
- HVM64
InstanceType: m1.small
```

The example template contains an AWS::EC2::Instance resource whose ImageId property is set by the FindInMap function.

MapName is set to the map of interest, "RegionMap" in this example. TopLevelKey is set to the region where the stack is created, which is determined by using the "AWS::Region" pseudo parameter.

SecondLevelKey is set to the desired architecture, "HVM64" for this example.

FindInMap returns the AMI assigned to FindInMap. For a HVM64 instance in us-east-1, FindInMap would return "ami-0ff8a91507f77f867".

Incorrect options:

!FindInMap [MapName, TopLevelKey]

!FindInMap [MapName]

!FindInMap [MapName, TopLevelKey, SecondLevelKey, ThirdLevelKey]

These three options contradict the explanation provided above, hence these options are incorrect.

Question 2

A company is creating a gaming application that will be deployed on mobile devices. The application will send data to a Lambda function-based RESTful API. The application will assign each API request a unique identifier. The volume of API requests from the application can randomly vary at any given time of day. During request throttling, the application might need to retry requests. The API must be able to address duplicate requests without inconsistencies or data loss.

Which of the following would you recommend to handle these requirements?

- Persist the unique identifier for each request in a DynamoDB table. Change the Lambda function to send a client error response when the function receives a duplicate request
- Persist the unique identifier for each request in an RDS MySQL table. Change the Lambda function to check the table for the identifier before processing the request
- Persist the unique identifier for each request in a DynamoDB table. Change the Lambda function to check the table for the identifier before processing the request
- Persist the unique identifier for each request in an ElastiCache for Memcached cache. Change the Lambda function to check the cache for the identifier before processing the request

Overall explanation

Correct option:

Persist the unique identifier for each request in a DynamoDB table. Change the Lambda function to check the table for the identifier before processing the request

DynamoDB is a fully managed, serverless, key-value NoSQL database designed to run high-performance applications at any scale. DynamoDB offers built-in security, continuous backups, automated multi-Region replication, in-memory caching, and data import and export tools. On-demand backup and restore allows you to create full backups of your DynamoDB. Point-in-time recovery (PITR) helps protect your DynamoDB tables from accidental write or delete operations. PITR provides continuous backups of your DynamoDB table data, and you can restore that table to any point in time up to the second during the preceding 35 days.

These features ensure that there is no data loss for the application, thereby meeting a key requirement for the given use case. The solution should also be able to address any duplicate requests without inconsistencies, so the Lambda function should be changed to inspect the table for the given identifier and process the request only if the identifier is unique.

DynamoDB Overview:



via - <https://aws.amazon.com/dynamodb/>

DynamoDB

Incorrect options:

Persist the unique identifier for each request in an ElastiCache for Memcached cache. Change the Lambda function to check the cache for the identifier before processing the request -

Memcached is designed for simplicity and it does not offer any snapshot or replication features. This can lead to data loss for applications. Therefore, this option is not the right fit for the given use case.

via - <https://aws.amazon.com/elasticache/redis-vs-memcached>

Persist the unique identifier for each request in an RDS MySQL table. Change the Lambda function to check the table for the identifier before processing the request - DynamoDB is a better fit than RDS MySQL to handle massive traffic spikes for write requests. DynamoDB is a key-value and document database that supports tables of virtually any size with horizontal scaling. DynamoDB scales to more than 10 trillion requests per day and with tables that have more than ten million read and write requests per second and petabytes of data storage. DynamoDB can be used to build applications that need consistent single-digit millisecond performance. MySQL RDS can be scaled vertically, however, it cannot match the performance benefits offered by DynamoDB for the given use case.

Question 3

- Use AWS Lambda Custom Authorizers
- Use AWS Billing Usage Plans
- Use API Gateway Usage Plans
- Use CloudFront Usage Plans

Use API Gateway Usage Plans

How API Gateway Works:



You can configure usage plans and API keys to allow customers to access selected APIs at agreed-upon request rates and quotas that meet their business requirements and budget constraints.

Overview of API Gateway Usage Plans and API keys:

What are usage plans and API keys?

API keys are alphanumeric string values that you distribute to application developer customers to grant access to your API. You can use API keys together with usage plans or Lambda authorizers to control access to your APIs. API Gateway can generate API keys on your behalf, or you can import them from a CSV file. You can generate an API key in API Gateway, or import it into API Gateway from an external source. For more information, see [Set up API keys using the API Gateway console](#).

Important

API key values must be unique. If you try to create two API keys with different names and the same value, API Gateway considers them to be the same API key.

An API key can be associated with more than one usage plan. A usage plan can be associated with more than one stage. However, a given API key can only be associated with one usage plan for each stage of your API.

A quota limit is the maximum number of requests with a given API key that can be submitted within a specified time interval. You can configure individual API methods to require API key authorization based on usage plan configuration. You can also use the `get-usage` CLI command or the `usage:get` REST API method to determine the usage for an API customer.

Note

Throttling and quota limits apply to requests for individual API keys that are aggregated across all API stages within a usage plan.

via - <https://docs.aws.amazon.com/apigateway/latest/developerguide/api-gateway-api-usage-plans.html>

Incorrect options:

Use AWS Billing Usage Plans - AWS Billing and Cost Management is the service that you use to pay your AWS bill, monitor your usage, and analyze and control your costs. There is no such thing as AWS Billing Usage Plans. You cannot use AWS Billing to set up public APIs for the application.

Use CloudFront Usage Plans - Amazon CloudFront is a fast content delivery network (CDN) service that securely delivers data, videos, applications, and APIs to customers globally with low latency.

high transfer speeds, all within a developer-friendly environment. There is no such thing as CloudFront Usage Plans. You cannot use CloudFront to set up public APIs for the application.

Use AWS Lambda Custom Authorizers - Lambda is a separate service than Gateway API, therefore, it cannot be used to determine the API usage limits.

Question 4

ECS Fargate container tasks are usually spread across Availability Zones (AZs) and the underlying workloads need persistent cross-AZ shared access to the data volumes configured for the container tasks.

Which of the following solutions is the best choice for these workloads?

- Docker volumes
- AWS Gateway Storage volumes
- Amazon EFS volumes
- Bind mounts

Overall explanation

Correct option:

Amazon EFS volumes - EFS volumes provide a simple, scalable, and persistent file storage for use with your Amazon ECS tasks. With Amazon EFS, storage capacity is elastic, growing and shrinking automatically as you add and remove files. Your applications can have the storage they need, when they need it. Amazon EFS volumes are supported for tasks hosted on Fargate or Amazon EC2 instances.

You can use Amazon EFS file systems with Amazon ECS to export file system data across your fleet of container instances. That way, your tasks have access to the same persistent storage, no matter the instance on which they land. However, you must configure your container instance AML to mount the Amazon EFS file system before the Docker daemon starts. Also, your task definitions must reference volume mounts on the container instance to use the file system.

Incorrect options:

Docker volumes - A Docker-managed volume that is created under `/var/lib/docker/volumes` on the host Amazon EC2 instance. Docker volume drivers (also referred to as plugins) are used to integrate the volumes with external storage systems, such as Amazon EBS. The built-in local volume driver or a third-party volume driver can be used. Docker volumes are only supported when running tasks on Amazon EC2 instances.

Bind mounts - A file or directory on the host, such as an Amazon EC2 instance or AWS Fargate, is mounted into a container. Bind mount host volumes are supported for tasks hosted on Fargate or Amazon EC2 instances. Bind mounts provide temporary storage, and hence these are a wrong choice for this use case.

AWS Storage Gateway volumes - This is an incorrect choice, given only as a distractor.

Question 5

Amazon Simple Queue Service (SQS) has a set of APIs for various actions supported by the service.

As a developer associate, which of the following would you identify as correct regarding the `CreateQueue` API? (Select two)

- The dead-letter queue of a FIFO queue must also be a FIFO queue. Whereas, the dead-letter queue of a standard queue can be a standard queue or a FIFO queue
- You can't change the queue type after you create it
- The length of time, in seconds, for which the delivery of all messages in the queue is delayed is configured using `MessageRetentionPeriod` attribute
- Queue tags are case insensitive. A new tag with a key identical to that of an existing tag overwrites the existing tag
- The visibility timeout value for the queue is in seconds, which defaults to 30 seconds

Overall explanation

Correct options:

You can't change the queue type after you create it - You can't change the queue type after you create it and you can't convert an existing standard queue into a FIFO queue. You must either create a new FIFO queue for your application or delete your existing standard queue and recreate it as a FIFO queue.

The visibility timeout value for the queue is in seconds, which defaults to 30 seconds - The visibility timeout for the queue is in seconds. Valid values are: An integer from 0 to 43,200 (12 hours), the Default value is 30.

Incorrect options:

The dead-letter queue of a FIFO queue must also be a FIFO queue. Whereas, the dead-letter queue of a standard queue can be a standard queue or a FIFO queue - The dead-letter queue of a FIFO queue must also be a FIFO queue. Similarly, the dead-letter queue of a standard queue must also be a standard queue.

The length of time, in seconds, for which the delivery of all messages in the queue is delayed is configured using `MessageRetentionPeriod` attribute - The length of time, in seconds, for which the delivery of all messages in the queue is delayed is configured using `DelaySeconds` attribute. `MessageRetentionPeriod` attribute controls the length of time, in seconds, for which Amazon SQS retains a message.

Queue tags are case insensitive. A new tag with a key identical to that of an existing tag overwrites the existing tag - Queue tags are case-sensitive. A new tag with a key identical to that of an existing tag overwrites the existing tag. To be able to tag a queue on creation, you must have the `sqs:CreateQueue` and `sqs:TagQueue` permissions.

Question 6

You have chosen AWS Elastic Beanstalk to upload your application code and allow it to handle details such as provisioning resources and monitoring.

When creating configuration files for AWS Elastic Beanstalk which naming convention should you follow?

- `.config/<mysettings>.ebextensions`
- `.ebextensions/<mysettings>.config`
- `.config_<mysettings>.ebextensions`
- `.ebextensions_<mysettings>.config`

Overall explanation

Correct option:

.ebextensions/<mysettings>.config : You can add AWS Elastic Beanstalk configuration files (`.ebextensions`) to your web application's source code to configure your environment and customize the AWS resources that it contains. Configuration files are YAML- or JSON-formatted documents with a `.config` file extension that you place in a folder named `.ebextensions` and deploy in your application source bundle.

Advanced environment customization with configuration files
(`.ebextensions`)

new | modify

You can add AWS Elastic Beanstalk configuration files (`.ebextensions`) to your web application's source code to configure your environment and customize the AWS resources that it contains. Configuration files are YAML- or JSON-formatted documents with a `.config` file extension that you place in a folder named `.ebextensions` and deploy in your application source bundle.

Example
`.ebextensions/network-load-balancer.config`

This example makes a simple configuration change. It modifies a configuration option to set the type of your environment's load balancer to Network Load Balancer.

```
option_settings:
  aws:elasticbeanstalk:environment:
    LoadBalancerType: network
```

We recommend using YAML for your configuration files, because it's more readable than JSON. YAML supports comments, multi-line commands, several alternatives for using quotes, and more. However, you can make any configuration change in Elastic Beanstalk configuration files identically using either YAML or JSON.



Tip

When you are developing or testing new configuration files, launch a clean environment running the default application and deploy to that. Poorly formatted configuration files will cause a new environment launch to fail unaccountably.

via - <https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/ebextensions.html>

Incorrect options:

`.ebextensions_<mysettings>.config`
`.config/<mysettings>.ebextensions`
`.config_<mysettings>.ebextensions`

These three options contradict the explanation provided earlier. So these are incorrect.

Question 7

An IT company is configuring Auto Scaling for its Amazon EC2 instances spread across different AZs and Regions.

Which of the following scenarios are NOT correct about EC2 Auto Scaling? (Select two)

- An Auto Scaling group can contain EC2 instances in one or more Availability Zones within the same Region
- Amazon EC2 Auto Scaling attempts to distribute instances evenly between the Availability Zones that are enabled for your Auto Scaling group
- For Auto Scaling groups in a VPC, the EC2 instances are launched in subnets
- Auto Scaling groups that span across multiple Regions need to be enabled for all the Regions specified
- An Auto Scaling group can contain EC2 instances in only one Availability Zone of a Region

Overall explanation

Correct options:

Auto Scaling groups that span across multiple Regions need to be enabled for all the Regions specified - This is not valid for Auto Scaling groups. Auto Scaling groups cannot span across multiple Regions.

An Auto Scaling group can contain EC2 instances in only one Availability Zone of a Region - This is not valid for Auto Scaling groups. An Auto Scaling group can contain EC2 instances in one or more Availability Zones within the same Region.

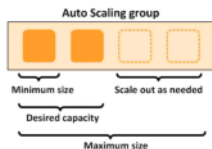
Amazon EC2 Auto Scaling Overview:

What Is Amazon EC2 Auto Scaling?

PDF | Kindle | RSS

Amazon EC2 Auto Scaling helps you ensure that you have the correct number of Amazon EC2 instances available to handle the load for your application. You create collections of EC2 instances, called Auto Scaling groups. You can specify the minimum number of instances in each Auto Scaling group, and Amazon EC2 Auto Scaling ensures that your group never goes below this size. You can specify the maximum number of instances in each Auto Scaling group, and Amazon EC2 Auto Scaling ensures that your group never goes above this size. If you specify the desired capacity, either when you create the group or at any time thereafter, Amazon EC2 Auto Scaling ensures that your group has this many instances. If you specify scaling policies, then Amazon EC2 Auto Scaling can launch or terminate instances as demand on your application increases or decreases.

For example, the following Auto Scaling group has a minimum size of one instance, a desired capacity of two instances, and a maximum size of four instances. The scaling policies that you define adjust the number of instances, within your minimum and maximum number of instances, based on the criteria that you specify.



via - <https://docs.aws.amazon.com/autoscaling/ec2/userguide/what-is-amazon-ec2-auto-scaling.html>

Incorrect options:

An Auto Scaling group can contain EC2 instances in one or more Availability Zones within the same Region - This is a valid statement. Auto Scaling groups can span across the availability Zones of a Region.

Amazon EC2 Auto Scaling attempts to distribute instances evenly between the Availability Zones that are enabled for your Auto Scaling group - When one Availability Zone becomes unhealthy or unavailable, Auto Scaling launches new instances in an unaffected Availability Zone. When the unhealthy Availability Zone returns to a healthy state, Auto Scaling automatically redistributes the application instances evenly across all of the designated Availability Zones.

For Auto Scaling groups in a VPC, the EC2 instances are launched in subnets - For Auto Scaling groups in a VPC, the EC2 instances are launched in subnets. Customers can select the subnets for your EC2 instances when you create or update the Auto Scaling group.

Question 8

A retail company is migrating its on-premises database to Amazon RDS for PostgreSQL. The company has read-heavy workloads. The development team at the company is looking at refactoring the code to achieve optimum read performance for SQL queries.

Which solution will address this requirement with the least current as well as future development effort?

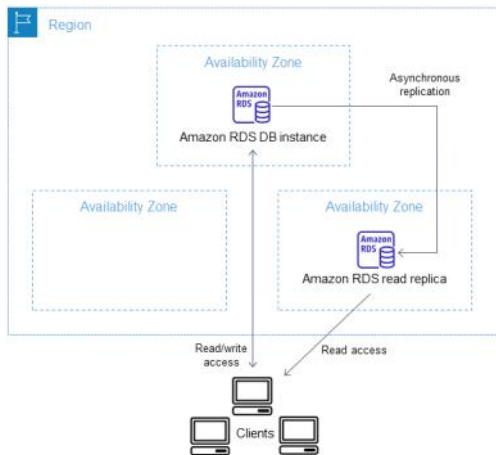
- Configure ElastiCache for Redis to act as a caching layer for Amazon RDS. Refactor the application code so that the queries use the ElastiCache for Redis endpoint
- Set up Amazon RDS with one or more read replicas. Refactor the application code so that the queries use the endpoint for the read replicas
- Set up Amazon RDS in the multi-AZ configuration with a single standby instance. Refactor the application code so that the queries use the standby instance endpoint
- Configure ElastiCache for Memcached to act as a caching layer for Amazon RDS. Refactor the application code so that the queries use the ElastiCache for Memcached endpoint

Overall explanation

Correct option:

Set up Amazon RDS with one or more read replicas. Refactor the application code so that the queries use the endpoint for the read replicas

Amazon RDS uses the PostgreSQL DB engine's built-in replication functionality to create a special type of DB instance called a read replica from a source DB instance. The source DB instance becomes the primary DB instance. Updates made to the primary DB instance are asynchronously copied to the read replica. You can reduce the load on your primary DB instance by routing read queries from your applications to the read replica. Using read replicas, you can elastically scale out beyond the capacity constraints of a single DB instance for read-heavy database workloads. For the given use case, you can achieve optimum read performance for SQL queries by using the read-replica endpoint for the read-heavy workload.



via - https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/USER_ReadRepl.html

Incorrect options:

Configure ElastiCache for Redis to act as a caching layer for Amazon RDS. Refactor the application code so that the queries use the ElastiCache for Redis endpoint

Configure ElastiCache for Memcached to act as a caching layer for Amazon RDS. Refactor the application code so that the queries use the ElastiCache for Memcached endpoint

Both Redis and Memcached are popular, open-source, in-memory data stores (also known as in-memory caches). These are not relational databases and cannot be used to run SQL queries. So, both these options are incorrect.

Set up Amazon RDS in the multi-AZ configuration with a single standby instance. Refactor the application code so that the queries use the standby instance endpoint - In an Amazon RDS Multi-AZ deployment with a single standby instance, Amazon RDS automatically creates a primary database (DB) instance and synchronously replicates the data to an instance in a different AZ. When it detects a failure, Amazon RDS automatically fails over to a standby instance without manual intervention. You cannot route the read queries from an application to the standby instance of a multi-AZ RDS database as it's not accessible for the read traffic in the single standby instance configuration.

Question 9

An application is hosted by a 3rd party and exposed at yourapp.3rdparty.com. You would like to have your users access your application using www.mydomain.com, which you own and manage under Route 53.

What Route 53 record should you create?

- Create a PTR record
- Create a CNAME record
- Create an Alias Record
- Create an A record

Overall explanation

Correct option:

Create a CNAME record

A CNAME record maps DNS queries for the name of the current record, such as acme.example.com, to another domain (example.com or example.net) or subdomain (acme.example.com or zenith.example.org).

CNAME records can be used to map one domain name to another. Although you should keep in mind that the DNS protocol does not allow you to create a CNAME record for the top node of a DNS namespace, also known as the zone apex. For example, if you register the DNS name example.com, the zone apex is example.com. You cannot create a CNAME record for example.com, but you can

create CNAME records for www.example.com, newproduct.example.com, and so on. Please review the major differences between CNAME and Alias Records:

Comparison of alias and CNAME records

Alias records are similar to CNAME records, but there are some important differences. The following list compares alias records and CNAME records.

Resources that you can redirect queries to

Alias records

An alias record can only redirect queries to selected AWS resources, such as the following:

- Amazon S3 buckets
- CloudFront distributions
- Another record in the same Route 53 hosted zone

For example, you can create an alias record named `acme.example.com` that redirects queries to an Amazon S3 bucket that is also named `acme.example.com`. You can also create an `acme.example.com` alias record that redirects queries to a record named `zereth.example.com` in the `example.com` hosted zone.

CNAME records

A CNAME record can redirect DNS queries to any DNS record. For example, you can create a CNAME record that redirects queries from `acme.example.com` to `zereth.example.com` or to `acme.example.org`. You don't need to use Route 53 as the DNS service for the domain that you're redirecting queries to.

Creating records that have the same name as the domain (records at the zone apex)

Alias records

In most configurations, you can create an alias record that has the same name as the hosted zone (the zone apex). The one exception is when you want to redirect queries from the zone apex (such as `example.com`) to a record in the same hosted zone that has a type of CNAME (such as `zereth.example.com`). The alias record must have the same type as the record you're routing traffic to, and creating a CNAME record for the zone apex isn't supported even for an alias record.

CNAME records

You can't create a CNAME record that has the same name as the hosted zone (the zone apex). This is true both for hosted zones for domain names (`example.com`) and for hosted zones for subdomains (`zereth.example.com`).

Pricing for DNS queries

Alias records

Route 53 doesn't charge for alias queries to AWS resources. For more information, see [Amazon Route 53 Pricing](#).

CNAME records

Route 53 charges for CNAME queries.

via - <https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/resource-record-sets-choosing-alias-non-alias.html>

Incorrect options:

Create an A record - Used to point a domain or subdomain to an IP address. 'A record' cannot be used to map one domain name to another.

Create a PTR record - A Pointer (PTR) record resolves an IP address to a fully-qualified domain name (FQDN) as an opposite to what A record does. PTR records are also called Reverse DNS records. 'PTR record' cannot be used to map one domain name to another.

Create an Alias Record - Alias records let you route traffic to selected AWS resources, such as CloudFront distributions and Amazon S3 buckets. They also let you route traffic from one record in a hosted zone to another record. 3rd party websites do not qualify for these as we have no control over those. 'Alias record' cannot be used to map one domain name to another.

Question 10

The technology team at an investment bank uses DynamoDB to facilitate high-frequency trading where multiple trades can try and update an item at the same time. Which of the following actions would make sure that only the last updated value of any item is used in the application?

- Use `ConsistentRead = true` while doing `PutItem` operation for any item
- Use `ConsistentRead = true` while doing `UpdateItem` operation for any item
- Use `ConsistentRead = false` while doing `PutItem` operation for any item
- Use `ConsistentRead = true` while doing `GetItem` operation for any item

Overall explanation
Correct option:
Use `ConsistentRead = true` while doing `GetItem` operation for any item
DynamoDB supports eventually consistent and strongly consistent reads.

Eventually Consistent Reads
When you read data from a DynamoDB table, the response might not reflect the results of a recently completed write operation. The response might include some stale data. If you repeat your read request after a short time, the response should return the latest data.

Strongly Consistent Reads
When you request a strongly consistent read, DynamoDB returns a response with the most up-to-date data, reflecting the updates from all prior write operations that were successful. DynamoDB uses eventually consistent reads by default. Read operations (such as `GetItem`, `Query`, and `Scan`) provide a `ConsistentRead` parameter. If you set this parameter to `true`, DynamoDB uses strongly consistent reads during the operation. As per the given use-case, to make sure that only the last updated value of any item is used in the application, you should use strongly consistent reads by setting `ConsistentRead = true` for `GetItem` operation.

Read Consistency

new | icons | rss

Amazon DynamoDB is available in multiple AWS Regions around the world. Each Region is independent and isolated from other AWS Regions. For example, if you have a table called `People` in the `us-east-2` Region and another table named `People` in the `us-west-2` Region, these are considered two entirely separate tables. For a list of all the AWS Regions in which DynamoDB is available, see [AWS Regions and Endpoints](#) in the [Amazon Web Services General Reference](#).

Every AWS Region consists of multiple distinct locations called **Availability Zones**. Each **Availability Zone** is isolated from failures in other **Availability Zones**, and provides inexpensive, low-latency network connectivity to other **Availability Zones** in the same Region. This allows rapid replication of your data among multiple **Availability Zones** in a Region.

When your application writes data to a DynamoDB table and receives an HTTP 200 response (OK), the write has occurred and is durable. The data is **eventually consistent** across all read operations, usually within one second or less.

DynamoDB supports **eventually consistent** and **strongly consistent reads**.

Eventually Consistent Reads

When you read data from a DynamoDB table, the response might not reflect the results of a recently completed write operation. The response might include some stale data. If you repeat your read request after a short time, the response should return the latest data.

Strongly Consistent Reads

When you request a strongly consistent read, DynamoDB returns a response with the most up-to-date data, reflecting the updates from all prior write operations that were successful. However, this consistency comes with some disadvantages:

- A strongly consistent read might not be available if there is a network delay or outage. In this case, DynamoDB may return a server error (HTTP 500).
- Strongly consistent reads may have higher latency than eventually consistent reads.
- Strongly consistent reads are not supported on global secondary indexes.
- Strongly consistent reads use more throughput capacity than eventually consistent reads. For details, see [Read/Write Capacity Mode](#).

Note
DynamoDB uses eventually consistent reads, unless you specify otherwise. Read operations such as `GetItem`, `Query`, and `Scan` provide a `ConsistentRead` parameter. If you set this parameter to `true`, DynamoDB uses strongly consistent reads during the operation.

via - <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.ReadConsistency.html>

Incorrect options:

Use `ConsistentRead = true` while doing `UpdateItem` operation for any item
Use `ConsistentRead = true` while doing `PutItem` operation for any item
Use `ConsistentRead = false` while doing `PutItem` operation for any item
As mentioned in the explanation above, strongly consistent reads apply only while using the read operations (such as `GetItem`, `Query`, and `Scan`). So these three options are incorrect.

Question 11

The development team at a HealthCare company has deployed EC2 instances in AWS Account A. These instances need to access patient data with Personally Identifiable Information (PII) on multiple S3 buckets in another AWS Account B.

As a Developer Associate, which of the following solutions would you recommend for the given use-case?

- Create an IAM role (instance profile) in Account A and set Account B as a trusted entity. Attach this role to the EC2 instances in Account A and add an inline policy to this role to access S3 data from Account B
- Add a bucket policy to all the Amazon S3 buckets in Account B to allow access from EC2 instances in Account A
- Copy the underlying AMI for the EC2 instances from Account A into Account B. Launch EC2 instances in Account B using this AMI and then access the PII data on Amazon S3 in Account B
- Create an IAM role with S3 access in Account B and set Account A as a trusted entity. Create another role (instance profile) in Account A and attach it to the EC2 instances in Account A and add an inline policy to this role to assume the role from Account B

Overall explanation
Correct option:
Create an IAM role with S3 access in Account B and set Account A as a trusted entity. Create another role (instance profile) in Account A and attach it to the EC2 instances in Account A and add an inline policy to this role to assume the role from Account B
You can give EC2 instances in one account ("account A") permissions to assume a role from another account ("account B") to access resources such as S3 buckets. You need to create an IAM role in Account B and set Account A as a trusted entity. Then attach a policy to this IAM role such that it delegates access to Amazon S3 like so -

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "s3:*",
      "Resource": [
```


Then you can create another role (instance profile) in Account A and attach it to the EC2 instances in Account A and add an inline policy to this role to assume the role from Account B like so -

Question 12

- Use AWS Glue to copy your table to Amazon S3 and download locally
- Use Hive with Amazon EMR to export your data to an S3 bucket and download locally
- Use the DynamoDB on-demand backup capability to write to Amazon S3 and download locally
- Use AWS Data Pipeline to export your table to an S3 bucket in the account of your choice and download locally

Correct option: **Use the DynamoDB on-demand backup capability to write to Amazon S3 and download locally** - This option is not feasible for the given use-case. DynamoDB has two built-in backup methods (On-demand, Point-in-time recovery) that write to Amazon S3, but you will not have access to the S3 buckets that are used for these backups.

via - <https://aws.amazon.com/premiumsupport/knowledge-center/back-up-dynamodb-s3/>

Question 13

☐ Neither Regional Reserved Instances nor Zonal Reserved Instances
☐ Both Regional Reserved Instances and Zonal Reserved Instances
☐ Zonal Reserved Instances
☐ Regional Reserved Instances

Regional and Zonal Reserved Instances:

Regional and zonal Reserved Instances (scope)

PDF | Kindle | RSS

When you purchase a Reserved Instance, you determine the scope of the Reserved Instance. The scope is either regional or zonal.

- **Regional:** When you purchase a Reserved Instance for a Region, it's referred to as a **regional Reserved Instance**.
- **Zonal:** When you purchase a Reserved Instance for a specific Availability Zone, it's referred to as a **zonal Reserved Instance**.

Differences between regional and zonal Reserved Instances

The following table highlights some key differences between regional Reserved Instances and zonal Reserved Instances:

	Regional Reserved Instances	Zonal Reserved Instances
Availability Zone Flexibility	The Reserved Instance discount applies to instance usage in any Availability Zone in the specified Region.	No Availability Zone flexibility—the Reserved Instance discount applies to instance usage in the specified Availability Zone only.
Capacity reservation	No capacity reservation—a regional Reserved Instance does not provide a capacity reservation.	A zonal Reserved Instance provides a capacity reservation in the specified Availability Zone.
Instance size flexibility	The Reserved Instance discount applies to instance usage within the instance family, regardless of size. Only supported on Amazon Linux/Ubuntu Reserved Instances with default tenancy. For more information, see Instance size flexibility determined by normalization factor .	No instance size flexibility—the Reserved Instance discount applies to instance usage for the specified instance type and size only.

via - <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/reserved-instances-scope.html>

High Level Overview of EC2 Instance Purchase Options:

On-Demand

With On-Demand instances, you pay for compute capacity by the hour or the second, depending on which instance you run. No long-term commitments or upfront payments are needed. You can increase or decrease your compute capacity depending on the demands of your application and only pay the specified per-hour rates for the instance you use.

On-Demand instances are recommended for:

- Cases that require the best cost and flexibility of Amazon EC2 without any up-front payment or long-term commitment.
- Applications with short-term, spiky, or unpredictable workloads that cannot be forecasted.
- Applications being developed or tested on Amazon EC2 for the first time.

[See On-Demand pricing](#)

Spot Instances

Amazon EC2 Spot Instances allow you to request spare Amazon EC2 computing capacity for as little as 90% off the On-Demand price. Look for more.

Spot instances are recommended for:

- Applications that have flexible start and end times.
- Applications that are only feasible at very low compute prices.
- Cases with urgent computing needs for large amounts of additional capacity.

[See Spot pricing](#)

Savings Plans

Savings Plans are a flexible pricing model that offer low prices on EC2 and Fargate usage, in exchange for a commitment to a consistent amount of usage (measured in Dollars) for a 1 or 3 year term.

Dedicated Hosts

A Dedicated Host is a physical EC2 server dedicated for your use. Dedicated Hosts can help you reduce costs by allowing you to use your existing server-based software licenses (including Windows Server, SQL Server, and SUSE Linux Enterprise Server) subject to your license terms, and also help you meet compliance requirements, learn more.

- Can be purchased On-Demand (hourly).
- Can be purchased as a Reservation for up to 70% off the On-Demand price.

[See Dedicated pricing](#)

Reserved Instances

Reserved Instances provide you with a significant discount (up to 70% compared to On-Demand instance pricing). In addition, when Reserved Instances are assigned to a specific Availability Zone, they provide a capacity reservation, giving you additional confidence in your ability to launch instances when you need them.

Reserved Instances are recommended for:

- Applications that have steady state or predictable usage. Reserved Instances can provide significant savings compared to using On-Demand instances. [See How to Purchase Reserved Instances](#) for more information.
- Reserved Instances are recommended for:
 - Applications with steady state usage.
 - Applications that may require reserved capacity.
 - Customers that can commit to using EC2 over a 1 or 3 year term to reduce their total computing costs.

via - <https://aws.amazon.com/ec2/pricing/>

Incorrect options:

Regional Reserved Instances - When you purchase a Reserved Instance for a Region, it's referred to as a regional Reserved Instance. A regional Reserved Instance does not provide a capacity reservation.

Both Regional Reserved Instances and Zonal Reserved Instances - As discussed above, only Zonal Reserved Instances provide capacity reservation.

Neither Regional Reserved Instances nor Zonal Reserved Instances - As discussed above, Zonal Reserved Instances provide capacity reservation.

Question 14

A business has their test environment built on Amazon EC2 configured on General purpose SSD volume.

At which gp2 volume size will their test environment hit the max IOPS?

- 5.3 TiB
- 2.7 TiB
- 16 TiB
- 10.6 TiB

Overall explanation

Correct option:

The performance of gp2 volumes is tied to volume size, which determines the baseline performance level of the volume and how quickly it accumulates I/O credits; larger volumes have higher baseline performance levels and accumulate I/O credits faster.

5.3 TiB - General Purpose SSD (gp2) volumes offer cost-effective storage that is ideal for a broad range of workloads. These volumes deliver single-digit millisecond latencies and the ability to burst to 3,000 IOPS for extended periods of time. Between a minimum of 100 IOPS (at 33.33 GiB and below) and a maximum of 16,000 IOPS (at 5,334 GiB and above), baseline performance scales linearly at 3 IOPS per GiB of volume size.

Maximum IOPS vs Volume Size for General Purpose SSD (gp2) volumes:

via - <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ebs-volume-types.html>

Incorrect options:

10.6 TiB - As explained above, this is an incorrect option.

16 TiB - As explained above, this is an incorrect option.

2.7 TiB - As explained above, this is an incorrect option.

Question 15

As a Senior Developer, you are tasked with creating several API Gateway powered APIs along with your team of developers. The developers are working on the API in the development environment, but they find the changes made to the APIs are not reflected when the API is called.

As a Developer Associate, which of the following solutions would you recommend for this use-case?

- Redeploy the API to an existing stage or to a new stage
- Developers need IAM permissions on API execution component of API Gateway
- Use Stage Variables for development state of API
- Enable Lambda authorizer to access API

Overall explanation

Correct option:

Redeploy the API to an existing stage or to a new stage

After creating your API, you must deploy it to make it callable by your users. To deploy an API, you create an API deployment and associate it with a stage. A stage is a logical reference to a lifecycle state of your API (for example, dev, prod, beta, v2). API stages are identified by the API ID and stage name. Every time you update an API, you must redeploy the API to an existing stage or to a new stage. Updating an API includes modifying routes, methods, integrations, authorizers, and anything else other than stage settings.

Incorrect options:

Developers need IAM permissions on API execution component of API Gateway - Access control access to Amazon API Gateway APIs is done with IAM permissions. To call a deployed API or to refresh the API caching, you must grant the API caller permissions to perform required IAM actions supported by the API execution component of API Gateway. In the current scenario, developers do not need permissions on "execution components" but on "management components" of API Gateway that help them to create, deploy, and manage an API. Hence, this statement is an incorrect option.

Enable Lambda authorizer to access API - A Lambda authorizer (formerly known as a custom authorizer) is an API Gateway feature that uses a Lambda function to control access to your API. So, this feature too helps in access control, but in the current scenario its the developers and not the users who are facing the issue. So, this statement is an incorrect option.

Use Stage Variables for development state of API - Stage variables are name-value pairs that you can define as configuration attributes associated with a deployment stage of a REST API. They act like environment variables and can be used in your API setup and mapping templates. Stage variables are not connected to the scenario described in the current use case.

Question 16

The development team at a multi-national retail company wants to support trusted third-party authenticated users from the supplier organizations to create and update records in specific DynamoDB tables in the company's AWS account.

As a Developer Associate, which of the following solutions would you suggest for the given use-case?

Your answer is incorrect

Use Cognito User pools to enable trusted third-party authenticated users to access DynamoDB

Create a new IAM group in the company's AWS account for each of the third-party authenticated users from the supplier organizations. The users can then use the IAM group credentials to access DynamoDB

Correct answer

Use Cognito Identity pools to enable trusted third-party authenticated users to access DynamoDB

Create a new IAM user in the company's AWS account for each of the third-party authenticated users from the supplier organizations. The users can then use the IAM user credentials to access DynamoDB

Overall explanation

Correct option:

Use Cognito Identity pools to enable trusted third-party authenticated users to access DynamoDB

Amazon Cognito identity pools (federated identities) enable you to create unique identities for your users and federate them with identity providers. With an identity pool, you can obtain temporary, limited-privilege AWS credentials to access other AWS services. Amazon Cognito identity pools support the following identity providers:

Public providers: Login with Amazon (Identity Pools), Facebook (Identity Pools), Google (Identity Pools), Sign in with Apple (Identity Pools).

Amazon Cognito User Pools

Open ID Connect Providers (Identity Pools)

SAML Identity Providers (Identity Pools)

Developer Authenticated Identities (Identity Pools)

Exam Alert:

Please review the following note to understand the differences between Cognito User Pools and Cognito Identity Pools:

Features of Amazon Cognito

User pools

A user pool is a user directory in Amazon Cognito. With a user pool, your users can sign in to your web or mobile app through Amazon Cognito, or federate through a third-party identity provider (IdP). Whether your users sign in directly or through a third party, all members of the user pool have a directory profile that you can access through an SDK.

User pools provider:

- Sign-up and sign-in services.
- A built-in, customizable web UI to sign in users.
- Social sign-in with Facebook, Google, Login with Amazon, and Sign in with Apple, and through SAML and OIDC identity providers from your user pool.
- User directory management and user profiles.
- Security features such as multi-factor authentication (MFA), checks for compromised credentials, account takeover protection, and phone and email verification.
- Customized workflows and user migration through AWS Lambda triggers.

For more information about user pools, see [Getting Started with User Pools](#) and the [Amazon Cognito User Pools API Reference](#).

Identity pools

With an identity pool, your users can obtain temporary AWS credentials to access AWS services, such as Amazon S3 and DynamoDB. Identity pools support anonymous guest users, as well as the following identity providers that you can use to authenticate users for identity pools:

- Amazon Cognito user pools
- Social sign-in with Facebook, Google, Login with Amazon, and Sign in with Apple
- OpenID Connect (OIDC) providers
- SAML identity providers
- Developer authenticated identities

To save user profile information, your identity pool needs to be integrated with a user pool.

For more information about identity pools, see [Getting Started with Amazon Cognito Identity Pools \(Federated Identities\)](#) and the [Amazon Cognito Identity Pools API Reference](#).

via - <https://docs.aws.amazon.com/cognito/latest/developerguide/what-is-amazon-cognito.html>

Incorrect options:

Use Cognito User pools to enable trusted third-party authenticated users to access DynamoDB - A user pool is a user directory in Amazon Cognito. With a user pool, your users can sign in to your web or mobile app through Amazon Cognito, or federate through a third-party identity provider (IdP). Cognito User Pools cannot be used to obtain temporary AWS credentials to access AWS services, such as Amazon S3 and DynamoDB.

Create a new IAM user in the company's AWS account for each of the third-party authenticated users from the supplier organizations. The users can then use the IAM user credentials to access DynamoDB

Create a new IAM group in the company's AWS account for each of the third-party authenticated users from the supplier organizations. The users can then use the IAM group credentials to access DynamoDB

Both these options involve setting up IAM resources such as IAM users or IAM groups just to provide access to DynamoDB tables. As the users are already trusted third-party authenticated users, Cognito Identity Pool can address this use-case in an elegant way.

Question 17

As a senior architect, you are responsible for the development, support, maintenance, and implementation of all database applications written using NoSQL technology. A new project demands a throughput requirement of 10 strongly consistent reads per second of 6KB in size each.

How many read capacity units will you need when configuring your DynamoDB table?

- 10
- 60
- 20
- 30

Overall explanation

Correct option:

Before proceeding with the calculations, please review the following:

Read Capacity Units

A read capacity unit represents one strongly consistent read per second, or two eventually consistent reads per second, for an item up to 4 KB in size.

Note

To learn more about DynamoDB read consistency models, see [Read Consistency](#).

For example, suppose that you create a table with 10 provisioned read capacity units. This allows you to perform 10 strongly consistent reads per second, or 20 eventually consistent reads per second, for items up to 4 KB.

Reading an item larger than 4 KB consumes more read capacity units. For example, a strongly consistent read of an item that is 8 KB (4 KB × 2) consumes 2 read capacity units. An eventually consistent read on that same item consumes only 1 read capacity unit.

Item sizes for reads are rounded up to the next 4 KB multiple. For example, reading a 3,500-byte item consumes the same throughput as reading a 4 KB item.

Capacity Unit Consumption for Reads

The following describes how DynamoDB read operations consume read capacity units:

- **GetItem**—Reads a single item from a table. To determine the number of capacity units that GetItem will consume, take the item size and round it up to the next 4 KB boundary. If you specified a strongly consistent read, this is the number of capacity units required. For an eventually consistent read (the default), divide this number by two.
For example, if you read an item that is 3.5 KB, DynamoDB rounds the item size to 4 KB. If you read an item of 10 KB, DynamoDB rounds the item size to 12 KB.
- **BatchGetItem**—Reads up to 100 items, from one or more tables. DynamoDB processes each item in the batch as an individual GetItem request, so DynamoDB first rounds up the size of each item to the next 4 KB boundary, and then calculates the total size. The result is not necessarily the same as the total size of all the items. For example, if BatchGetItem reads a 1.5 KB item and a 6.5 KB item, DynamoDB calculates the size as 12 KB (4 KB + 8 KB), not 8 KB (1.5 KB + 6.5 KB).
- **Query**—Reads multiple items that have the same partition key value. All items returned are treated as a single read operation, where DynamoDB computes the total size of all items and then rounds up to the next 4 KB boundary. For example, suppose your query returns 10 items whose combined size is 40.8 KB. DynamoDB rounds the item size for the operation to 44 KB. If a query returns 1500 items of 64 bytes each, the cumulative size is 96 KB.
- **Scan**—Reads all items in a table. DynamoDB considers the size of the items that are evaluated, not the size of the items returned by the scan.

Write Capacity Units

A write capacity unit represents one write per second, for an item up to 1 KB in size.

For example, suppose that you create a table with 10 write capacity units. This allows you to perform 10 writes per second, for items up to 1 KB in size per second.

Item sizes for writes are rounded up to the next 1 KB multiple. For example, writing a 500-byte item consumes the same throughput as writing a 1 KB item.

Capacity Unit Consumption for Writes

The following describes how DynamoDB write operations consume write capacity units:

- **PutItem**—Writes a single item to a table. If an item with the same primary key exists in the table, the operation replaces the item. For calculating provisioned throughput consumption, the item size that matters is the larger of the two.
- **UpdateItem**—Modifies a single item in the table. DynamoDB considers the size of the item as it appears before and after the update. The provisioned throughput consumption reflects the larger of those item sizes. Even if you update just a subset of the item's attributes, UpdateItem will still consume the full amount of provisioned throughput (the larger of the "before" and "after" item sizes).
- **DeleteItem**—Removes a single item from a table. The provisioned throughput consumption is based on the size of the deleted item.
- **BatchWriteItem**—Writes up to 25 items to one or more tables. DynamoDB processes each item in the batch as an individual PutItem or DeleteItem request (updates are not supported). So DynamoDB first rounds up the size of each item to the next 1 KB boundary, and then calculates the total size. The result is not necessarily the same as the total size of all the items. For example, if BatchWriteItem writes a 100-byte item and a 5.5 KB item, DynamoDB calculates the size as 5 KB (1 KB + 4 KB), not 4 KB (100 bytes + 5.5 KB).

For PutItem, UpdateItem, and DeleteItem operations, DynamoDB rounds the item size up to the next 1 KB. For example, if you put or delete an item of 1.6 KB, DynamoDB rounds the item size up to 2 KB.

via - <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/ProvisionedThroughput.html>

20

One read capacity unit represents one strongly consistent read per second for an item up to 4 KB in size. If you need to read an item that is larger than 4 KB, DynamoDB will need to consume additional read capacity units.

1) Item Size / 4KB, rounding to the nearest whole number.

So, in the above case, 6KB / 4 KB = 1.5 or 2 read capacity units.

2) 1 read capacity unit per item (since strongly consistent read) × No of reads per second

So, in the above case, 2 × 10 = 20 read capacity units.

Incorrect options:

60
30
10

These three options contradict the details provided in the explanation above, so these are incorrect.

Question 18

A pharmaceutical company runs their database workloads on Provisioned IOPS SSD (io1) volumes.

As a Developer Associate, which of the following options would you identify as an **INVALID** configuration for io1 EBS volume types?

- 200 GiB size volume with 15000 IOPS
- 200 GiB size volume with 5000 IOPS
- 200 GiB size volume with 10000 IOPS
- 200 GiB size volume with 2000 IOPS

Overall explanation

Correct option:

200 GiB size volume with 15000 IOPS - This is an invalid configuration. The maximum ratio of provisioned IOPS to requested volume size (in GiB) is 50:1. So, for a 200 GiB volume size, max IOPS possible is $200 \times 50 = 10000$ IOPS.

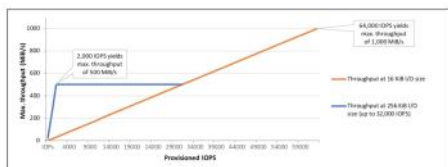
Overview of Provisioned IOPS SSD (io1) volumes:

Provisioned IOPS SSD (io1) volumes

Provisioned IOPS SSD (io1) volumes are designed to meet the needs of I/O-intensive workloads, particularly database workloads, that are sensitive to storage performance and consistency. Unlike gp2, which uses a bucket and credit model to calculate performance, an io1 volume allows you to specify a consistent IOPS rate when you create the volume, and Amazon EBS delivers the provisioned performance 99.9 percent of the time.

An io1 volume can range in size from 4 GiB to 16 TiB. You can provision from 100 IOPS up to 64,000 IOPS per volume on instances built on the Nitro System and up to 32,000 on other instances. The maximum ratio of provisioned IOPS to requested volume size (in GiB) is 50:1. For example, a 100 GiB volume can be provisioned with up to 5,000 IOPS. On a supported instance type, any volume 1,280 GiB in size or greater allows provisioning up to the 64,000 IOPS maximum ($50 \times 1,280 \text{ GiB} = 64,000$).

An io1 volume provisioned with up to 32,000 IOPS supports a maximum I/O size of 256 KiB and yields as much as 500 MiB/s of throughput. With the I/O size at the maximum, peak throughput is reached at 2,000 IOPS. A volume provisioned with more than 32,000 IOPS (up to the cap of 64,000 IOPS) supports a maximum I/O size of 16 KiB and yields as much as 1,000 MiB/s of throughput. The following graph illustrates these performance characteristics:



via - <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ebs-volume-types.html>

Incorrect options:

Provisioned IOPS SSD (io1) volumes allow you to specify a consistent IOPS rate when you create the volume, and Amazon EBS delivers the provisioned performance 99.9 percent of the time. An io1 volume can range in size from 4 GiB to 16 TiB. The maximum ratio of provisioned IOPS to the requested volume size (in GiB) is 50:1. For example, a 100 GiB volume can be provisioned with up to 5,000 IOPS.

200 GiB size volume with 2000 IOPS - As explained above, up to 10000 IOPS is a valid configuration for the given use-case.

200 GiB size volume with 10000 IOPS - As explained above, up to 10000 IOPS is a valid configuration for the given use-case.

200 GiB size volume with 5000 IOPS - As explained above, up to 10000 IOPS is a valid configuration for the given use-case.

Question 19

As an AWS certified developer associate, you are working on an AWS CloudFormation template that will create resources for a company's cloud infrastructure. Your template is composed of three stacks which are Stack-A, Stack-B, and Stack-C. Stack-A will provision a VPC, a security group, and subnets for public web applications that will be referenced in Stack-B and Stack-C. After running the stacks you decide to delete them, in which order should you do it?

- Stack A, Stack C then Stack B
- Stack A, then Stack B, then Stack C
- Stack B, then Stack C, then Stack A
- Stack C then Stack A then Stack B

Overall explanation

Correct option:

AWS CloudFormation gives developers and businesses an easy way to create a collection of related AWS and third-party resources and provision them in an orderly and predictable fashion.

How CloudFormation Works:



via - <https://aws.amazon.com/cloudformation/>

Stack B, then Stack C, then Stack A

All of the imports must be removed before you can delete the exporting stack or modify the output value. In this case, you must delete Stack B as well as Stack C, before you delete Stack A.

Exporting stack output values

PDF Kindle RSS

To share information between stacks, export a stack's output values. Other stacks that are in the same AWS account and region can import the exported values. For example, you might have a single networking stack that exports the IDs of a subnet and security group for public web servers. Stacks with a public web server can easily import those networking resources. You don't need to hard code resource IDs in the stack's template or pass IDs as input parameters.

To export a stack's output value, use the `Export` field in the `Output` section of the stack's template. To import those values, use the `Fn::ImportValue` function in the template for the other stacks. For a walkthrough and sample templates, see [Walkthrough: Refer to resource outputs in another AWS CloudFormation stack](#).

Note

After another stack imports an output value, you can't delete the stack that is exporting the output value or modify the exported output value. All of the imports must be removed before you can delete the exporting stack or modify the output value.

via - <https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/using-cfn-stack-exports.html>

Incorrect options:

Stack A, then Stack B, then Stack C - All of the imports must be removed before you can delete the exporting stack or modify the output value. In this case, you cannot delete Stack A first because that's being referenced in the other stacks.

Stack A, Stack C then Stack B - All of the imports must be removed before you can delete the exporting stack or modify the output value. In this case, you cannot delete Stack A first because that's being referenced in the other stacks.

Stack C then Stack A then Stack B - Stack C is fine but you should delete Stack B before Stack A because all of the imports must be removed before you can delete the exporting stack or modify the output value.

Reference: <https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/using-cfn-stack-exports.html>

Question 20

A developer working with EC2 Windows instance has installed Kinesis Agent for Windows to stream JSON-formatted log files to Amazon Simple Storage Service (S3) via Amazon Kinesis Data Firehose. The developer wants to understand the sink type capabilities of Kinesis Firehose. Which of the following sink types is NOT supported by Kinesis Firehose.

- Amazon Simple Storage Service (Amazon S3) as a direct Firehose destination
- Amazon Redshift with Amazon S3
- Amazon Elasticsearch Service (Amazon ES) with optionally backing up data to Amazon S3
- Amazon ElastiCache with Amazon S3 as backup

Overall explanation

Correct option:

Amazon Kinesis Data Firehose is a fully managed service for delivering real-time streaming data to destinations such as Amazon Simple Storage Service (Amazon S3), Amazon Redshift, Amazon Elasticsearch Service (Amazon ES), and Splunk. With Kinesis Data Firehose, you don't need to write applications or manage resources. You configure your data producers to send data to Kinesis Data Firehose, and it automatically delivers the data to the destination that you specified.

Amazon ElastiCache with Amazon S3 as backup - Amazon ElastiCache is a fully managed in-memory data store, compatible with Redis or Memcached. ElastiCache is NOT a supported destination for Amazon Kinesis Data Firehose.

Incorrect options:

Amazon Elasticsearch Service (Amazon ES) with optionally backing up data to Amazon S3 - Amazon ES is a supported destination type for Kinesis Firehose. Streaming data is delivered to your Amazon ES cluster, and can optionally be backed up to your S3 bucket concurrently.

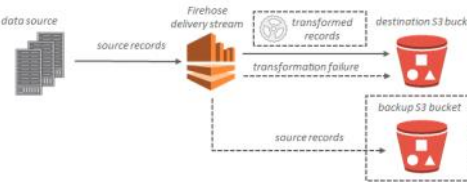
Data Flow for ES:



via - <https://docs.aws.amazon.com/firehose/latest/dev/what-is-this-service.html>

Amazon Simple Storage Service (Amazon S3) as a direct Firehose destination - For Amazon S3 destinations, streaming data is delivered to your S3 bucket. If data transformation is enabled, you can optionally back up source data to another Amazon S3 bucket.

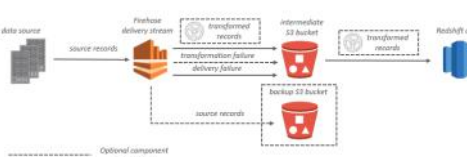
Data Flow for S3:



via - <https://docs.aws.amazon.com/firehose/latest/dev/what-is-this-service.html>

Amazon Redshift with Amazon S3 - For Amazon Redshift destinations, streaming data is delivered to your S3 bucket first. Kinesis Data Firehose then issues an Amazon Redshift COPY command to load data from your S3 bucket to your Amazon Redshift cluster. If data transformation is enabled, you can optionally back up source data to another Amazon S3 bucket.

Data Flow for Redshift:



via - <https://docs.aws.amazon.com/firehose/latest/dev/what-is-this-service.html>

Question 21

You have been asked by your Team Lead to enable detailed monitoring of the Amazon EC2 instances your team uses. As a Developer working on AWS CLI, which of the below command will you run?

- `aws ec2 monitor-instances --instance-ids i-1234567890abcdef0`
- `aws ec2 run-instances --image-id ami-09092360 --monitoring Enabled=true`
- `aws ec2 monitor-instances --instance-id i-1234567890abcdef0`
- `aws ec2 run-instances --image-id ami-09092360 --monitoring State=enabled`

Overall explanation

Correct option:

aws ec2 monitor-instances --instance-ids i-1234567890abcdef0 - This enables detailed monitoring for a running instance.

EC2 detailed monitoring:

```
monitor-instances
--instance-ids <value>
[--dry-run | --no-dry-run]
[--cli-input-json <value>]
[--generate-cli-skeleton <value>]
```

Options

--instance-ids (list)

The IDs of the instances.

(string)

Syntax:

```
"string" "string" ...
```

--dry-run | --no-dry-run (boolean)

Checks whether you have the required permissions for the action, without actually making the request, and provides an error response. If you have the required permissions, the error response is `DryRunOperation`. Otherwise, it is `UnauthorizedOperation`.

--cli-input-json (string) Performs service operation based on the JSON string provided. The JSON string follows the format provided by `--generate-cli-skeleton`. If other arguments are provided on the command line, the CLI values will override the JSON-provided values. It is not possible to pass arbitrary binary values using a JSON-provided value as the string will be taken literally.

--generate-cli-skeleton (string) Prints a JSON skeleton to standard output without sending an API request. If provided with no value or the value `input`, prints a sample input JSON that can be used as an argument for `--cli-input-json`. If provided with the value `output`, it validates the command inputs and returns a sample output JSON for that command.

via - <https://docs.aws.amazon.com/cli/latest/reference/ec2/monitor-instances.html>

Incorrect options:

aws ec2 run-instances --image-id ami-09092360 --monitoring Enabled=true - This syntax is used to enable detailed monitoring when launching an instance from AWS CLI.
aws ec2 run-instances --image-id ami-09092360 --monitoring State=enabled - This is an invalid syntax
aws ec2 monitor-instances --instance-id i-1234567890abcdef0 - This is an invalid syntax

Question 22

You have migrated an on-premise SQL Server database to an Amazon Relational Database Service (RDS) database attached to a VPC inside a private subnet. Also, the related Java application, hosted on-premise, has been moved to an Amazon Lambda function.
Which of the following should you implement to connect AWS Lambda function to its RDS instance?

- Configure Lambda to connect to VPC with private subnet and Security Group needed to access RDS
- Configure lambda to connect to the public subnet that will give internet access and use Security Group to access RDS inside the private subnet
- Use Lambda layers to connect to the internet and RDS separately
- Use Environment variables to pass in the RDS connection string

Overall explanation

Correct option:

Configure Lambda to connect to VPC with private subnet and Security Group needed to access RDS - You can configure a Lambda function to connect to private subnets in a virtual private cloud (VPC) in your account. Use Amazon Virtual Private Cloud (Amazon VPC) to create a private network for resources such as databases, cache instances, or internal services. Connect your lambda function to the VPC to access private resources during execution. When you connect a function to a VPC, Lambda creates an elastic network interface for each combination of the security group and subnet in your function's VPC configuration. This is the right way of giving RDS access to Lambda.

Lambda VPC Config:



via - <https://docs.aws.amazon.com/lambda/latest/dg/configuration-vpc.html>

Incorrect options:

Use Lambda layers to connect to the internet and RDS separately - You can configure your Lambda function to pull in additional code and content in the form of layers. A layer is a ZIP archive that contains libraries, a custom runtime, or other dependencies. Layers will not help in configuring access to RDS instance and hence is an incorrect choice.
Configure lambda to connect to the public subnet that will give internet access and use the Security Group to access RDS inside the private subnet - This is an incorrect statement. Connecting a Lambda function to a public subnet does not give it internet access or a public IP address. To grant internet access to your function, its associated VPC must have a NAT gateway (or NAT instance) in a public subnet.
Use Environment variables to pass in the RDS connection string - You can use environment variables to store secrets securely and adjust your function's behavior without updating code. You can use environment variables to exchange data with RDS, but you will still need access to RDS, which is not possible with just environment variables.

Question 23

You have a three-tier web application consisting of a web layer using AngularJS, an application layer using an AWS API Gateway and a data layer in an Amazon Relational Database Service (RDS) database. Your web application allows visitors to look up popular movies from the past. The company is looking at reducing the number of calls made to endpoint and improve latency to the API.
What can you do to improve performance?

- Use Amazon Kinesis Data Streams to stream incoming data and reduce the burden on Gateway APIs
- Enable API Gateway Caching
- Use Stage Variables
- Use Mapping Templates

Overall explanation

Correct option:

Enable API Gateway Caching - You can enable API caching in Amazon API Gateway to cache your endpoint's responses. With caching, you can reduce the number of calls made to your endpoint and also improve the latency of requests to your API. When you enable caching for a stage, API Gateway caches responses from your endpoint for a specified time-to-live (TTL) period, in seconds. API Gateway then responds to the request by looking up the endpoint response from the cache instead of making a request to your endpoint. The default TTL value for API caching is 300 seconds. The maximum TTL value is 3600 seconds. TTL=0 means caching is disabled.

Incorrect options:

Use Mapping Templates - A mapping template is a script expressed in Velocity Template Language (VTL) and applied to the payload using JSONPath expressions. Mapping templates help format/structure the data in a way that it is easily readable, unlike a server response that might always be easy to ready. Mapping Templates do not help in latency issues of the APIs.
Use Stage Variables - Stage variables act like environment variables and can be used to change the behavior of your API Gateway methods for each deployment stage; for example, making it possible to reach a different back end depending on which stage the API is running on. Stage variables do not help in latency issues.
Use Amazon Kinesis Data Streams to stream incoming data and reduce the burden on Gateway APIs - Amazon Kinesis Data Streams (KDS) is a massively scalable and durable real-time data streaming service. KDS can continuously capture gigabytes of data per second from hundreds of thousands of sources such as website clickstreams, database event streams, financial transactions, social media feeds, IT logs, and location-tracking events.

Question 24

A media company uses Amazon Simple Queue Service (SQS) queue to manage their transactions. With changing business needs, the payload size of the messages is increasing. The Team Lead of the project is worried about the 256 KB message size limit that SQS has.
What can be done to make the queue accept messages of a larger size?

- Get a service limit increase from AWS
- Use the SQS Extended Client
- Use gzip compression
- Use the MultiPart API

Overall explanation

Correct option:

Use the SQS Extended Client - To manage large Amazon Simple Queue Service (Amazon SQS) messages, you can use Amazon Simple Storage Service (Amazon S3) and the Amazon SQS Extended Client Library for Java. This is especially useful for storing and consuming messages up to 2 GB. Unless your application requires repeatedly creating queues and leaving them inactive or storing large amounts of data in your queues, consider using Amazon S3 for storing your data.

Incorrect options:

Use the MultiPart API - This is an incorrect statement. There is no multi-part API for Amazon Simple Queue Service.
Get a service limit increase from AWS - While it is possible to get service limits extended for certain AWS services, AWS already offers Extended Client to deal with queues that have larger messages.
Use gzip compression - You can compress the messages before sending them to the queue. The messages also need to be encoded after this to cater to SQS message standards. This adds bulk to the messages and will not be an optimal solution for the current scenario.

Question 25

Your web application reads and writes data to your DynamoDB table. The table is provisioned with 400 Write Capacity Units (WCU's) shared across 4 partitions. One of the partitions receives 250 WCU/second while others receive much less. You receive the error 'ProvisionedThroughputExceededException'.
What is the likely cause of this error?

- CloudWatch monitoring is lagging
- Configured IAM policy is wrong
- You have a hot partition
- Write Capacity Units (WCU's) are applied across to all your DynamoDB tables and this needs reconfiguration

Overall explanation

Correct option:

You have a hot partition

It's not always possible to distribute read and write activity evenly. When data access is imbalanced, a "hot" partition can receive a higher volume of read and write traffic compared to other partitions. To better accommodate uneven access patterns, DynamoDB adaptive capacity enables your application to continue reading and writing to hot partitions without being throttled, provided that traffic does not exceed your table's total provisioned capacity or the partition maximum capacity.

ProvisionedThroughputExceededException explained:

Request Throttling and Burst Capacity

If your application performs reads or writes at a higher rate than your table can support, DynamoDB begins to throttle those requests. When DynamoDB throttles a read or write, it returns a **ProvisionedThroughputExceededException** to the caller. The application can then take appropriate action, such as waiting for a short interval before retrying the request.



Note

We recommend that you use the AWS SDKs for software development. The AWS SDKs provide built-in support for retrying throttled requests; you do not need to write this logic yourself. For more information, see [Error Resilience and Exponential Backoff](#).

The DynamoDB console displays Amazon CloudWatch metrics for your tables, so you can monitor throttled read requests and write requests. If you encounter excessive throttling, you should consider increasing your table's provisioned throughput settings.

In some cases, DynamoDB uses burst capacity to accommodate reads or writes in excess of your table's throughput settings. With burst capacity, unexpected read or write requests can succeed where they otherwise would be throttled. For more information, see [Using Burst Capacity Effectively](#).

via - <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/ProvisionedThroughput.html>

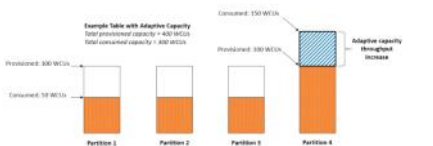
Hot partition explained:

Boost Throughput Capacity to High-Traffic Partitions

It's not always possible to distribute read and write activity evenly. When data access is imbalanced, a "hot" partition can receive a higher volume of read and write traffic compared to other partitions. In extreme cases, throttling can occur if a single partition receives more than 5,000 RCUs or 1,000 WCUs.

To better accommodate uneven access patterns, DynamoDB adaptive capacity enables your application to continue reading and writing to hot partitions without being throttled, provided that traffic does not exceed your table's total provisioned capacity or the partition maximum capacity. Adaptive capacity works by automatically and instantly increasing throughput capacity for partitions that receive more traffic.

The following diagram illustrates how adaptive capacity works. The example table is provisioned with 400 WCUs evenly shared across four partitions, allowing each partition to sustain up to 100 WCUs per second. Partitions 1, 2, and 3 each receive write traffic of 50 WCUs/sec. Partition 4 receives 150 WCUs/sec. This hot partition can sustain write traffic while it still has unused burst capacity, but eventually it throttles traffic that exceeds 100 WCUs/sec. DynamoDB adaptive capacity responds by increasing partition 4's capacity so that it can sustain the higher workload of 150 WCUs/sec without being throttled.



via - <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/bp-partition-key-design.html>

Incorrect options:

CloudWatch monitoring is lagging - The error is specific to DynamoDB itself and not to any connected service. CloudWatch is a fully managed service from AWS and does not result in throttling.

Configured IAM policy is wrong - The error is not associated with authorization but to exceeding something pre-configured value. So, it's clearly not a permissions issue.

Write-capacity units (WCU's) are applied across to all your DynamoDB tables and this needs reconfiguration - This statement is incorrect. Read Capacity Units and Write Capacity Units are specific to one table.

Question 26

A leading financial services company offers data aggregation services for Wall Street trading firms. The company bills its clients based on per unit of clickstream data provided to the clients. As the company operates in a regulated industry, it needs to have the same ordered clickstream data available for auditing within a window of 7 days.

As a Developer Associate, which of the following AWS services do you think provides the ability to run the billing process and auditing process on the given clickstream data in the same order?

- AWS Kinesis Data Firehose
- Amazon SQS
- AWS Kinesis Data Analytics
- AWS Kinesis Data Streams

Overall explanation

Correct option:

AWS Kinesis Data Streams

Amazon Kinesis Data Streams (KDS) is a massively scalable and durable real-time data streaming service. KDS can continuously capture gigabytes of data per second from hundreds of thousands of sources such as website clickstreams, database event streams, financial transactions, social media feeds, IT logs, and location-tracking events. The data collected is available in milliseconds to enable real-time analytics use cases such as real-time dashboards, real-time anomaly detection, dynamic pricing, and more.

Amazon Kinesis Data Streams enables real-time processing of streaming big data. It provides ordering of records, as well as the ability to read and/or replay records in the same order to multiple Amazon Kinesis Applications. The Amazon Kinesis Client Library (KCL) delivers all records for a given partition key to the same record processor, making it easier to build multiple applications reading from the same Amazon Kinesis data stream (for example, to perform counting, aggregation, and filtering). Amazon Kinesis Data Streams is recommended when you need the ability to consume records in the same order a few hours later.

For example, you have a billing application and an audit application that runs a few hours behind the billing application. By default, records of a stream are accessible for up to 24 hours from the time they are added to the stream. You can raise this limit to a maximum of 365 days. For the given use-case, Amazon Kinesis Data Streams can be configured to store data for up to 7 days and you can run the audit application up to 7 days behind the billing application.

KDS provides the ability to consume records in the same order a few hours later

via - <https://aws.amazon.com/kinesis/data-streams/faqs/>

Incorrect options:

AWS Kinesis Data Firehose - Amazon Kinesis Data Firehose is the easiest way to load streaming data into data stores and analytics tools. It can capture, transform, and load streaming data into Amazon S3, Amazon Redshift, Amazon Elasticsearch Service, and Splunk, enabling near real-time analytics with existing business intelligence tools and dashboards you're already using today. It is a fully managed service that automatically scales to match the throughput of your data and requires no ongoing administration. It can also batch, compress, and encrypt the data before loading it, minimizing the amount of storage used at the destination and increasing security. As Kinesis Data Firehose is used to load streaming data into data stores, therefore this option is incorrect.

AWS Kinesis Data Analytics - Amazon Kinesis Data Analytics is the easiest way to analyze streaming data in real-time. You can quickly build SQL queries and sophisticated Java applications using built-in templates and operators for common processing functions to organize, transform, aggregate, and analyze data at any scale. Kinesis Data Analytics enables you to easily and quickly build queries and sophisticated streaming applications in three simple steps: setup your streaming data sources, write your queries or streaming applications and set up your destination for processed data. As Kinesis Data Analytics is used to build SQL queries and sophisticated Java applications, therefore this option is incorrect.

Amazon SQS - Amazon Simple Queue Service (SQS) is a fully managed message queuing service that enables you to decouple and scale microservices, distributed systems, and serverless applications. SQS offers two types of message queues. Standard queues offer maximum throughput, best-effort ordering, and at-least-once delivery. SQS FIFO queues are designed to guarantee that messages are processed exactly once, in the exact order that they are sent. For SQS, you cannot have the same message being consumed by multiple consumers in the same order a few hours later, therefore this option is incorrect.

References:

<https://aws.amazon.com/kinesis/data-streams/faqs/>

<https://aws.amazon.com/kinesis/data-firehose/faqs/>

<https://aws.amazon.com/kinesis/data-analytics/faqs/>

Question 27

The development team at a company wants to encrypt a 111 GB object using AWS KMS. Which of the following represents the best solution?

- Make an Encrypt API call to encrypt the plaintext data as ciphertext using a customer master key (CMK) with imported key material
- Make a GenerateDataKeyWithoutPlaintext API call that returns an encrypted copy of a data key. Use an encrypted key to encrypt the data
- Make a GenerateDataKeyWithPlaintext API call that returns an encrypted copy of a data key. Use a plaintext key to encrypt the data
- Make a GenerateDataKey API call that returns a plaintext key and an encrypted copy of a data key. Use a plaintext key to encrypt the data

Overall explanation

Correct option:

Make a GenerateDataKey API call that returns a plaintext key and an encrypted copy of a data key. Use a plaintext key to encrypt the data - GenerateDataKey API, generates a unique symmetric data key for client-side encryption. This operation returns a plaintext copy of the data key and a copy that is encrypted under a customer master key (CMK) that you specify. You can use the plaintext key to encrypt your data outside of AWS KMS and store the encrypted data key with the encrypted data.

GenerateDataKey returns a unique data key for each request. The bytes in the plaintext key are not related to the caller or the CMK.

To encrypt data outside of AWS KMS:

1. Use the GenerateDataKey operation to get a data key.
2. Use the plaintext data key (in the Plaintext field of the response) to encrypt your data outside of AWS KMS. Then erase the plaintext data key from memory.
3. Store the encrypted data key (in the CiphertextBlob field of the response) with the encrypted data.

To decrypt data outside of AWS KMS:

4. Use the Decrypt operation to decrypt the encrypted data key. The operation returns a plaintext copy of the data key.
5. Use the plaintext data key to decrypt data outside of AWS KMS, then erase the plaintext data key from memory.

Incorrect options:

Make a GenerateDataKeyWithPlaintext API call that returns an encrypted copy of a data key. Use a plaintext key to encrypt the data - This is a made-up option, given only as a distractor.

Make an Encrypt API call to encrypt the plaintext data as ciphertext using a customer master key (CMK) with imported key material - Encrypt API is used to encrypt plaintext into ciphertext by using a customer master key (CMK). The Encrypt operation has two primary use cases:

6. To encrypt small amounts of arbitrary data, such as a personal identifier or database password, or other sensitive information.
7. To move encrypted data from one AWS Region to another.

Neither of the two is useful for the given scenario.

Make a GenerateDataKeyWithoutPlaintext API call that returns an encrypted copy of a data key. Use an encrypted key to encrypt the data - GenerateDataKeyWithoutPlaintext API, generates a unique symmetric data key. This operation returns a data key that is encrypted under a customer master key (CMK) that you specify.

GenerateDataKeyWithoutPlaintext is identical to the GenerateDataKey operation except that returns only the encrypted copy of the data key. This operation is useful for systems that need to encrypt data at some point, but not immediately. When you need to encrypt the data, you call the Decrypt operation on the encrypted copy of the key.

Question 28

To meet compliance guidelines, a company needs to ensure replication of any data stored in its S3 buckets. Which of the following characteristics are correct while configuring an S3 bucket for replication? (Select two)

- Replicated objects do not retain metadata
- Same-Region Replication (SRR) and Cross-Region Replication (CRR) can be configured at the S3 bucket level, a shared prefix level, or an object level using S3 object tags
- Object tags cannot be replicated across AWS Regions using Cross-Region Replication
- Once replication is enabled on a bucket, all old and new objects will be replicated
- S3 lifecycle actions are not replicated with S3 replication

Overall explanation

Correct options:

Same-Region Replication (SRR) and Cross-Region Replication (CRR) can be configured at the S3 bucket level, a shared prefix level, or an object level using S3 object tags - Amazon S3 Replication (CRR and SRR) is configured at the S3 bucket level, a shared prefix level, or an object level using S3 object tags. You add a replication configuration on your source bucket by specifying a destination bucket in the same or different AWS region for replication.

S3 lifecycle actions are not replicated with S3 replication - With S3 Replication (CRR and SRR), you can establish replication rules to make copies of your objects into another storage class, in the same or a different region. Lifecycle actions are not replicated, and if you want the same lifecycle configuration applied to both source and destination buckets, enable the same lifecycle configuration on both.

Incorrect options:

Object tags cannot be replicated across AWS Regions using Cross-Region Replication - Object tags can be replicated across AWS Regions using Cross-Region Replication. For customers with Cross-Region Replication already enabled, new permissions are required for tags to replicate.

Once replication is enabled on a bucket, all old and new objects will be replicated - Replication only replicates the objects added to the bucket after replication is enabled on the bucket. Any objects present in the bucket before enabling replication are not replicated.

Replicated objects do not retain metadata - You can use replication to make copies of your objects that retain all metadata, such as the original object creation time and version IDs. This capability is important if you need to ensure that your replica is identical to the source object.

Question 29

A development team has created a new IAM user that has s3:putObject permission to write to an S3 bucket. This S3 bucket uses server-side encryption with AWS KMS managed keys (SSE-KMS) as the default encryption. Using the access key ID and the secret access key of the IAM user, the application received an access denied error when calling the PutObject API.

As a Developer Associate, how would you resolve this issue?

- Correct the bucket policy of the S3 bucket to allow the IAM user to upload encrypted objects
- Correct the ACL of the S3 bucket to allow the IAM user to upload encrypted objects
- Correct the policy of the IAM user to allow the s3:Encrypt action
- Correct the policy of the IAM user to allow the kms:GenerateDataKey action

Overall explanation

Correct option:

Correct the policy of the IAM user to allow the kms:GenerateDataKey action - You can protect data at rest in Amazon S3 by using three different modes of server-side encryption: SSE-S3, SSE-C, or SSE-KMS. SSE-KMS requires that AWS manage the data key but you manage the customer master key (CMK) in AWS KMS. You can choose a customer managed CMK or the AWS managed CMK for Amazon S3 in your account. If you choose to encrypt your data using the standard features, AWS KMS and Amazon S3 perform the following actions:

8. Amazon S3 requests a plaintext data key and a copy of the key encrypted under the specified CMK.
9. AWS KMS generates a data key, encrypts it under the CMK, and sends both the plaintext data key and the encrypted data key to Amazon S3.
10. Amazon S3 encrypts the data using the data key and removes the plaintext key from memory as soon as possible after use.
11. Amazon S3 stores the encrypted data key as metadata with the encrypted data.

The error message indicates that your IAM user or role needs permission for the kms:GenerateDataKey action. This permission is required for buckets that use default encryption with a custom AWS KMS key.

In the JSON policy documents, look for policies related to AWS KMS access. Review statements with "Effect": "Allow" to check if the user or role has permissions for the kms:GenerateDataKey action on the bucket's AWS KMS key. If this permission is missing, then add the permission to the appropriate policy.

In the JSON policy documents, look for statements with "Effect": "Deny". Then, confirm that those statements don't deny the s3:PutObject action on the bucket. The statements must also not deny the IAM user or role access to the kms:GenerateDataKey action on the key used to encrypt the bucket. Additionally, make sure the necessary KMS and S3 permissions are not restricted using a VPC endpoint policy, service control policy, permissions boundary, or session policy.

Incorrect options:

Correct the policy of the IAM user to allow the s3:Encrypt action - This is an invalid action given only as a distractor.

Correct the bucket policy of the S3 bucket to allow the IAM user to upload encrypted objects - The user already has access to the bucket. What the user lacks is access to generate a KMS key, which is mandatory when a bucket is enabled for default encryption.

Correct the ACL of the S3 bucket to allow the IAM user to upload encrypted objects - Amazon S3 access control lists (ACLs) enable you to manage access to buckets and objects. Each bucket and object has an ACL attached to it as a subresource. It defines which AWS accounts or groups are granted access and the type of access. ACL is another way of giving access to S3 bucket objects. Permissions to use KMS keys will still be needed.

Question 30

A team is checking the viability of using AWS Step Functions for creating a banking workflow for loan approvals. The web application will also have human approval as one of the steps in the workflow. As a developer associate, which of the following would you identify as the key characteristics for AWS Step Function? (Select two)

- You should use Express Workflows for workloads with high event rates and short duration
- Standard Workflows on AWS Step Functions are suitable for long-running, durable, and auditable workflows that do not support any human approval steps
- Express Workflows have a maximum duration of five minutes and Standard workflows have a maximum duration of 180 days or 6 months
- Both Standard and Express Workflows support all service integrations, activities, and design patterns
- Standard Workflows on AWS Step Functions are suitable for long-running, durable, and auditable workflows that can also support any human approval steps

Overall explanation

Correct options:

Standard Workflows on AWS Step Functions are suitable for long-running, durable, and auditable workflows that can also support any human approval steps - Standard Workflows on AWS Step Functions are more suitable for long-running, durable, and auditable workflows where repeating workflow steps is expensive (e.g., restarting a long-running media transcode) or harmful (e.g., charging a credit card twice). Example workloads include training and deploying machine learning models, report generation, billing, credit card processing, and ordering and fulfillment processes. Step functions also support any human approval steps.

You should use Express Workflows for workloads with high event rates and short duration* - You should use Express Workflows for workloads with high event rates and short durations. Express Workflows support event rates of more than 100,000 per second.

Incorrect options:

Standard Workflows on AWS Step Functions are suitable for long-running, durable, and auditable workflows that do not support any human approval steps - As Step functions support any human approval steps, so this option is incorrect.

Express Workflows have a maximum duration of five minutes and Standard workflows have a maximum duration of 180 days or 6 months - Express Workflows have a maximum duration of five minutes and Standard workflows have a maximum duration of one year.

Both Standard and Express Workflows support all service integrations, activities, and design patterns - Standard Workflows support all service integrations, activities, and design patterns. Express Workflows do not support activities, job-run (.sync), and Callback patterns.

Question 31

A company's e-commerce website is expecting hundreds of thousands of visitors on Black Friday. The marketing department is concerned that high volumes of orders might stress SQS leading to message failures. The company has approached you for the steps to be taken as a precautionary measure against the high volumes. What step will you suggest as a Developer Associate?

- Enable auto-scaling in the SQS queue
- Pre-configure the SQS queue to increase the capacity when messages hit a certain threshold
- Amazon SQS is highly scalable and does not need any intervention to handle the expected high volumes
- Convert the queue into FIFO ordered queue, since messages to the down system will be processed faster once they are ordered

Overall explanation

Correct option:

Amazon SQS is highly scalable and does not need any intervention to handle the expected high volumes

Amazon SQS leverages the AWS cloud to dynamically scale, based on demand. SQS scales elastically with your application so you don't have to worry about capacity planning and pre-provisioning. For most standard queues (depending on queue traffic and message backlog), there can be a maximum of approximately 120,000 inflight messages (received from a queue by a consumer, but not yet deleted from the queue).

Info on Queue Quotas:

Quotas related to queues

The following table lists quotas related to queues.

Quota	Description
per queue (backlog)	
Messages per queue (in flight)	For most standard queues (depending on queue traffic and message backlog), there can be a maximum of approximately 120,000 inflight messages (received from a queue by a consumer, but not yet deleted from the queue). If you reach this quota while using short polling , Amazon SQS returns the OverLimit error message. If you use long polling , Amazon SQS returns no error messages. To avoid reaching the quota, you should delete messages from the queue after they're processed. You can also increase the number of queues you use to process your messages. To request a quota increase, submit a support request .
	For FIFO queues, there can be a maximum of 20,000 inflight messages (received from a queue by a consumer, but not yet deleted from the queue). If you reach this quota, Amazon SQS returns no error messages.
Queue	A queue name can have up to 80 characters. The following characters are

via - <https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-quotas.html>

Incorrect options:

Pre-configure the SQS queue to increase the capacity when messages hit a certain threshold - This is an incorrect statement. Amazon SQS scales dynamically, automatically provisioning the needed capacity.

Enable auto-scaling in the SQS queue - SQS queues are, by definition, auto-scalable and do not need any configuration changes for auto-scaling.

Convert the queue into FIFO ordered queue, since messages to the down system will be processed faster once they are ordered - This is a wrong statement. You cannot convert an existing standard queue to FIFO queue. To make the move, you must either create a new FIFO queue for your application or delete your existing standard queue and recreate it as a FIFO queue.

Standard to FIFO queue conversion:

Moving from a Standard queue to a FIFO queue

If you have an existing application that uses standard queues and you want to take advantage of the ordering or exactly-once processing features of FIFO queues, you need to configure the queue and your application correctly.

Note

You can't **convert** an existing standard queue into a FIFO queue. To make the move, you must either create a new FIFO queue for your application or delete your existing standard queue and recreate it as a FIFO queue.

via - <https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/FIFO-queues.html>

Question 32

Your company has a three-year contract with a healthcare provider. The contract states that monthly database backups must be retained for the duration of the contract for compliance purposes. Currently, the limit on backup retention for automated backups, on Amazon Relational Database Service (RDS), does not meet your requirements. Which of the following solutions can help you meet your requirements?

- Enable RDS automatic backups
- Create a cron event in CloudWatch, which triggers an AWS Lambda function that triggers the database snapshot
- Enable RDS Read replicas
- Enable RDS Multi-AZ

Overall explanation

Correct option:

Create a cron event in CloudWatch, which triggers an AWS Lambda function that triggers the database snapshot - There are multiple ways to run periodic jobs in AWS. Amazon EventBridge with Lambda is the simplest of all solutions. To do this, create a CloudWatch Rule and select "Schedule" as the Event Source. You can either use a cron expression or provide a fixed rate (such as every 5 minutes). Next, select "Lambda Function" as the Target. Your Lambda will have the necessary code for snapshot functionality.

Incorrect options:

Enable RDS automatic backups - You can enable automatic backups but as of 2020, the retention period is 0 to 35 days.

Enable RDS Read replicas - Amazon RDS server's built-in replication functionality to create a special type of DB instance called a read replica from a source DB instance. Updates made to the source DB instance are asynchronously copied to the read replica. Read replicas are useful for heavy read-only data workloads. These are not suitable for the given use-case.

Enable RDS Multi-AZ - Multi-AZ allows you to create a highly available application with RDS. It does not directly help in database backups or retention periods.

Question 33

You are a developer working with the AWS CLI to create Lambda functions that contain environment variables. Your functions will require over 50 environment variables consisting of sensitive information of database table names.

What is the total set size/number of environment variables you can create for AWS Lambda?

- The total size of all environment variables shouldn't exceed 4 KB. The maximum number of variables that can be created is 35
- The total size of all environment variables shouldn't exceed 8 KB. There is no limit on the number of variables
- The total size of all environment variables shouldn't exceed 4 KB. There is no limit on the number of variables
- The total size of all environment variables shouldn't exceed 8 KB. The maximum number of variables that can be created is 50

Overall explanation

Correct option:

The total size of all environment variables shouldn't exceed 4 KB. There is no limit on the number of variables

An environment variable is a pair of strings that are stored in a function's version-specific configuration. The Lambda runtime makes environment variables available to your code and sets additional environment variables that contain information about the function and invocation request. The total size of all environment variables doesn't exceed 4 KB. There is no limit defined on the number of variables that can be used.

Incorrect options:

The total size of all environment variables shouldn't exceed 8 KB. The maximum number of variables that can be created is 50 - Incorrect option. The total size of environment variables cannot exceed 4 KB with no restriction on the number of variables.

The total size of all environment variables shouldn't exceed 8 KB. There is no limit on the number of variables - Incorrect option. The total size of environment variables cannot exceed 4 KB with no restriction on the number of variables.

The total size of all environment variables shouldn't exceed 4 KB. The maximum number of variables that can be created is 35 - Incorrect option. The total size of environment variables cannot exceed 4 KB with no restriction on the number of variables.

Question 34

A new member of your team is working on creating Dead Letter Queue (DLQ) for AWS Lambda functions.

As a Developer Associate, can you help him identify the use cases, wherein AWS Lambda will add a message into a DLQ after being processed? (Select two)

- The event has been processed successfully
- The Lambda function invocation is synchronous
- The Lambda function invocation failed only once but succeeded thereafter
- The Lambda function invocation is asynchronous
- The event fails all processing attempts

Overall explanation

Correct option:

The Lambda function invocation is asynchronous - When an asynchronous invocation event exceeds the maximum age or fails all retry attempts, Lambda discards it. Or sends it to dead-letter queue if you have configured one.

The event fails all processing attempt - A dead-letter queue acts the same as an on-failure destination in that it is used when an event fails all processing attempts or expires without being processed.

Incorrect options:

The Lambda function invocation is synchronous - When you invoke a function synchronously, Lambda runs the function and waits for a response. Queues are generally used with asynchronous invocations since queues implement the decoupling feature of various connected systems. It does not make sense to use queues when the calling code will wait on it for a response.

The event has been processed successfully - A successfully processed event is not sent to the dead-letter queue.

The event processing failed only once but succeeded thereafter - A successfully processed event is not sent to the dead-letter queue.

Question 35

A development team uses shared Amazon S3 buckets to upload files. Due to this shared access, objects in S3 buckets have different owners making it difficult to manage the objects.

As a developer associate, which of the following would you suggest to automatically make the S3 bucket owner, also the owner of all objects in the bucket, irrespective of the AWS account used for uploading the objects?

- Use S3 CORS to make the S3 bucket owner, the owner of all objects in the bucket
- Use S3 Access Analyzer to identify the owners of all objects and change the ownership to the bucket owner
- Use Bucket Access Control Lists (ACLs) to control access on S3 bucket and then define its owner
- Use S3 Object Ownership to default bucket owner to be the owner of all objects in the bucket

Overall explanation

Correct option:

Use S3 Object Ownership to default bucket owner to be the owner of all objects in the bucket

S3 Object Ownership is an Amazon S3 bucket setting that you can use to control ownership of new objects that are uploaded to your buckets. By default, when other AWS accounts upload objects to your bucket, the objects remain owned by the uploading account. With S3 Object Ownership, any new objects that are written by other accounts with the bucket-owner-full-control canned access control list (ACL) automatically become owned by the bucket owner, who then has full control of the objects.

S3 Object Ownership has two settings: 1. Object writer – The uploading account will own the object. 2. Bucket owner preferred – The bucket owner will own the object if the object is uploaded with the bucket-owner-full-control canned ACL. Without this setting and canned ACL, the object is uploaded and remains owned by the uploading account.

Incorrect options:

Use S3 CORS to make the S3 bucket owner, the owner of all objects in the bucket - Cross-origin resource sharing (CORS) defines a way for client web applications that are loaded in one domain to interact with resources in a different domain.

Use S3 Access Analyzer to identify the owners of all objects and change the ownership to the bucket owner - Access Analyzer for S3 helps review all buckets that have bucket access control lists (ACLs), bucket policies, or access point policies that grant public or shared access. Access Analyzer for S3 alerts you to buckets that are configured to allow access to anyone on the internet or other AWS accounts, including AWS accounts outside of your organization.

Use Bucket Access Control Lists (ACLs) to control access on S3 bucket and then define its owner - Amazon S3 access control lists (ACLs) enable you to manage access to buckets and objects. Each bucket and object has an ACL attached to it as a subresource. A bucket ACLs allow you to control access at bucket level.

None of the above features are useful for the current scenario and hence are incorrect options.

Question 36

You have an Amazon Kinesis Data Stream with 10 shards, and from the metrics, you are well below the throughput utilization of 10 MB per second to send data. You send 3 MB per second of data and yet you are receiving ProvisionedThroughputExceededException errors frequently.

What is the likely cause of this?

Metrics are slow to update

You have too many shards

The partition key that you have selected isn't distributed enough

The data retention period is too long

Overall explanation

Correct option:

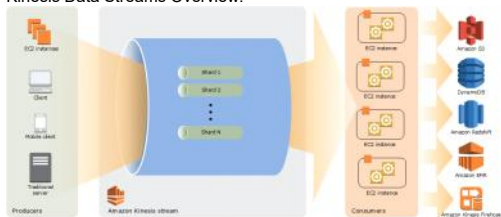
The partition key that you have selected isn't distributed enough

Amazon Kinesis Data Streams enables you to build custom applications that process or analyze streaming data for specialized needs.

A Kinesis data stream is a set of shards. A shard is a uniquely identified sequence of data records in a stream. A stream is composed of one or more shards, each of which provides a fixed unit of capacity.

The partition key is used by Kinesis Data Streams to distribute data across shards. Kinesis Data Streams segregates the data records that belong to a stream into multiple shards, using the partition key associated with each data record to determine the shard to which a given data record belongs.

Kinesis Data Streams Overview:



via - <https://docs.aws.amazon.com/streams/latest/dev/key-concepts.html>

For the given use-case, as the partition key is not distributed enough, all the data is getting skewed at a few specific shards and not leveraging the entire cluster of shards.

You can also use metrics to determine which are your "hot" or "cold" shards, that is, shards that are receiving much more data, or much less data, than expected. You could then selectively split the hot shards to increase capacity for the hash keys that target those shards. Similarly, you could merge cold shards to make better use of their unused capacity.

Incorrect options:

Metrics are slow to update - Metrics are a CloudWatch concept. This option has been added as a distractor.

You have too many shards - Too many shards is not the issue as you would see a LimitExceededException in that case.

The data retention period is too long - Your streaming data is retained for up to 365 days. The data retention period is not an issue causing this error.

References:

<https://docs.aws.amazon.com/streams/latest/dev/key-concepts.html>

<https://docs.aws.amazon.com/streams/latest/dev/kinesis-using-sdk-java-resharding-strategies.html>

Question 37

A .NET developer team works with many ASP.NET web applications that use EC2 instances to host them on IIS. The deployment process needs to be configured so that multiple versions of the application can run in AWS Elastic Beanstalk. One version would be used for development, testing, and another version for load testing. Which of the following methods do you recommend?

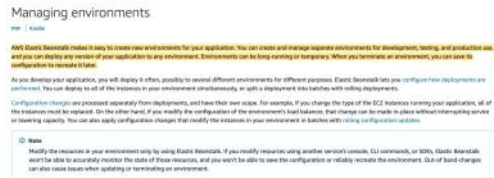
- You cannot have multiple development environments in Elastic Beanstalk, just one development and one production environment
- Define a dev environment with a single instance and a 'load test' environment that has settings close to production environment
- Create an Application Load Balancer to route based on hostname so you can pass on parameters to the development Elastic Beanstalk environment. Create a file in .ebextensions/ to know how to handle the traffic coming from the ALB
- Use only one Beanstalk environment and perform configuration changes using an Ansible script

Overall explanation

Correct option:

Define a dev environment with a single instance and a 'load test' environment that has settings close to production environment

AWS Elastic Beanstalk makes it easy to create new environments for your application. You can create and manage separate environments for development, testing, and production use, and you can deploy any version of your application to any environment. Environments can be long-running or temporary. When you terminate an environment, you can save its configuration to recreate it later. It is common practice to have many environments for the same application. You can deploy multiple environments when you need to run multiple versions of an application. So for the given use-case, you can set up 'dev' and 'load test' environment.



You cannot have multiple development environments in Elastic Beanstalk, just one development, and one production environment - Incorrect, use the Create New Environment wizard in the AWS Management Console for BeanStalk to guide you on this.

Use only one Beanstalk environment and perform configuration changes using an Ansible script - Ansible is an open-source deployment tool that integrates with AWS. It allows us to deploy the infrastructure. Elastic Beanstalk provisions the servers that you need for hosting the application and it also handles multiple environments, so Beanstalk is a better option.

Create an Application Load Balancer to route based on hostname so you can pass on parameters to the development Elastic Beanstalk environment. Create a file in .ebextensions/ to know how to handle the traffic coming from the ALB - This is not a good design if you need to load test because you will have two versions on the same instances and may not be able to access resources in the system due to the load testing.

Question 38

A firm maintains a highly available application that receives HTTPS traffic from mobile devices and web browsers. The main Developer would like to set up the Load Balancer routing to route traffic from web servers to smart.com/api and from mobile devices to smart.com/mobile. A developer advises that the previous recommendation is not needed and that requests should be sent to api.smart.com and mobile.smart.com instead.

Which of the following routing options were discussed in the given use-case? (select two)

- Client IP
- Web browser version
- Path based
- Host based
- Cookie value

Overall explanation

Correct options:

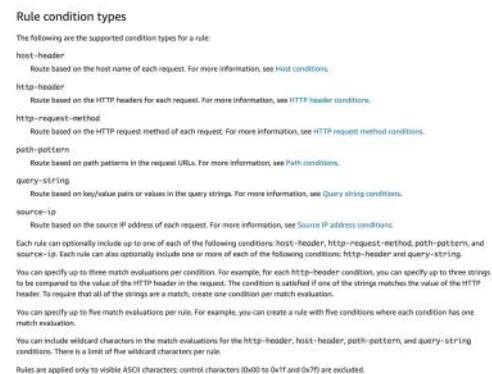
Path based

You can create a listener with rules to forward requests based on the URL path. This is known as path-based routing. If you are running microservices, you can route traffic to multiple back-end services using path-based routing. For example, you can route general requests to one target group and request to render images to another target group.

This path-based routing allows you to route requests to, for example, /api to one set of servers (also known as target groups) and /mobile to another set. Segmenting your traffic in this way gives you the ability to control the processing environment for each category of requests. Perhaps /api requests are best processed on Compute Optimized instances, while /mobile requests are best handled by Memory Optimized instances.

Host based

You can create Application Load Balancer rules that route incoming traffic based on the domain name specified in the Host header. Requests to api.example.com can be sent to one target group, requests to mobile.example.com to another, and all others (by way of a default rule) can be sent to a third. You can also create rules that combine host-based routing and path-based routing. This would allow you to route requests to api.example.com/production and api.example.com/sandbox to distinct target groups.



Incorrect options:

Client IP - This option has been added as a distractor. Routing is not based on the client's IP address.

Web browser version - Routing has nothing to do with the client's web browser, if it was then there is something sneaky going on.

Cookie value - Application Load Balancers support load balancer-generated cookies only and you cannot modify them. When routing sticky sessions to route requests to the same target then cookies are needed to be supported by the client's browser.

Question 39

You are assigned as the new project lead for a web application that processes orders for customers. You want to integrate event-driven processing anytime data is modified or deleted and use a serverless approach using AWS Lambda for processing stream events. Which of the following databases should you choose from?

- ElastiCache
- Kinesis
- RDS
- DynamoDB

Overall explanation

Correct option:

DynamoDB

A DynamoDB stream is an ordered flow of information about changes to items in a DynamoDB table. When you enable a stream on a table, DynamoDB Streams captures a time-ordered sequence of item-level modifications in any DynamoDB table, and stores this information in a log for up to 24 hours. Applications can access this log and view the data items as they appeared before and after they were modified, in near real-time. Whenever an application creates, updates, or deletes items in the table, DynamoDB Streams writes a stream record with the primary key attributes of the items that were modified. DynamoDB Streams Overview:

Capturing Table Activity with DynamoDB Streams

PDF | Kindle | RSS

Many applications can benefit from the ability to capture changes to items stored in a DynamoDB table, at the point in time when such changes occur. The following are some example use cases:

- An application in one AWS Region modifies the data in a DynamoDB table. A second application in another Region reads these data modifications and writes the data to another table, creating a replica that stays in sync with the original table.
- A popular mobile app modifies data in a DynamoDB table, at the rate of thousands of updates per second. Another application captures and stores data about these updates, providing near-real-time usage metrics for the mobile app.
- A global multi-player game has a multi-master topology, storing data in multiple AWS Regions. Each master stays in sync by consuming and replicating the changes that occur in the remote Regions.
- An application automatically sends notifications to the mobile devices of all friends in a group as soon as one friend uploads a new picture.
- A new customer adds data to a DynamoDB table. This event invokes another application that sends a welcome email to the new customer.

DynamoDB Streams enables solutions such as these, and many others. **DynamoDB Streams captures a time-ordered sequence of item-level modifications in any DynamoDB table and stores this information in a log for up to 24 hours.** Applications can access this log and view the data items as they appeared before and after they were modified, in near-real time.

Encryption at rest encrypts the data in DynamoDB streams. For more information, see [DynamoDB Encryption at Rest](#).

A DynamoDB stream is an ordered flow of information about changes to items in a DynamoDB table. When you enable a stream on a table, DynamoDB captures information about every modification to data items in the table.

Whenever an application creates, updates, or deletes items in the table, DynamoDB Streams writes a stream record with the primary key attributes of the items that were modified. A stream record contains information about a data modification to a single item in a DynamoDB table. You can configure the stream so that the stream records capture additional information, such as the "before" and "after" images of modified items.

DynamoDB Streams helps ensure the following:

- Each stream record appears exactly once in the stream.
- For each item that is modified in a DynamoDB table, the stream records appear in the same sequence as the actual modifications to the item.

DynamoDB Streams writes stream records in near-real time so that you can build applications that consume these streams and take action based on the contents.

via - <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Streams.html>

Incorrect options:

RDS - By itself, RDS cannot be used to stream events like DynamoDB, so this option is ruled out. However, you can use Amazon Kinesis for streaming data from RDS. Please refer to this excellent blog for more details on using Kinesis for streaming data from RDS: <https://aws.amazon.com/blogs/database/streaming-changes-in-a-database-with-amazon-kinesis/>

ElastiCache - ElastiCache works as an in-memory data store and cache, it cannot be used to stream data like DynamoDB.

Kinesis - Kinesis is not a database, so this option is ruled out.

Amazon Kinesis Data Streams enables you to build custom applications that process or analyze streaming data for specialized needs. You can continuously add various types of data such as clickstreams, application logs, and social media to an Amazon Kinesis data stream from hundreds of thousands of sources.

How Kinesis Data Streams Work



via - <https://aws.amazon.com/kinesis/data-streams/>

Question 40

Your company has been hired to build a resilient mobile voting app for an upcoming music award show that expects to have 5 to 20 million viewers. The mobile voting app will be marketed heavily months in advance so you are expected to handle millions of messages in the system. You are configuring Amazon Simple Queue Service (SQS) queues for your architecture that should receive messages from 20 KB to 200 KB. Is it possible to send these messages to SQS?

- Yes, the max message size is 512KB
- Yes, the max message size is 256KB
- No, the max message size is 64KB
- No, the max message size is 128KB

Overall explanation
Correct option:
Yes, the max message size is 256KB
The minimum message size is 1 byte (1 character). The maximum is 262,144 bytes (256 KB).

Q: How do I configure the maximum message size for Amazon SQS?

To configure the maximum message size, use the console or the `SetQueueAttributes` method to set the `MaximumMessageSize` attribute. This attribute specifies the number of bytes that an Amazon SQS message can contain. **Set this attribute to a value between 1,024 bytes (1 KB), and 262,144 bytes (256 KB).** For more information, see [Using Amazon SQS Message Attributes in the Amazon SQS Developer Guide](#).

To send messages larger than 256 KB, use the [Amazon SQS Extended Client Library for Java](#). This library lets you send an Amazon SQS message that contains a reference to a message payload in Amazon S3 that can be as large as 2 GB.

Q: What kind of data can I include in a message?

Amazon SQS messages can contain up to 256 KB of text data, including XML, JSON and unformatted text. The following Unicode characters are accepted:

`#x9 | #xA | #xD | [0x20 to 0xD7FF] | [0xE000 to 0xFFFF] | [0x10000 to 0x10FFFF]`

For more information, see the [XML 1.0 Specification](#).

via - <https://aws.amazon.com/sqs/faqs/>

Incorrect options:
Yes, the max message size is 512KB - The max size is 256KB
No, the max message size is 128KB - The max size is 256KB
No, the max message size is 64KB - The max size is 256KB

Question 41

You are designing a high-performance application that requires millions of connections. You have several EC2 instances running Apache2 web servers and the application will require capturing the user's source IP address and source port without the use of X-Forwarded-For. Which of the following options will meet your needs?

- Application Load Balancer
- Elastic Load Balancer
- Classic Load Balancer
- Network Load Balancer

Overall explanation
Correct option:
Network Load Balancer

A Network Load Balancer functions at the fourth layer of the Open Systems Interconnection (OSI) model. It can handle millions of requests per second. After the load balancer receives a connection request, it selects a target from the target group for the default rule. It attempts to open a TCP connection to the selected target on the port specified in the listener configuration. Incoming connections remain unmodified, so application software need not support X-Forwarded-For.

Network Load Balancer overview

A Network Load Balancer functions at the fourth layer of the Open Systems Interconnection (OSI) model. It can handle millions of requests per second. After the load balancer receives a connection request, it selects a target from the target group for the default rule. It attempts to open a TCP connection to the selected target on the port specified in the listener configuration.

When you enable an Availability Zone for the load balancer, Elastic Load Balancing creates a load balancer node in the Availability Zone. By default, each load balancer node distributes traffic across the registered targets in its Availability Zone only. If you enable cross-zone load balancing, each load balancer node distributes traffic across the registered targets in all enabled Availability Zones. For more information, see [Availability Zones](#).

If you enable multiple Availability Zones for your load balancer and ensure that each target group has at least one target in each enabled Availability Zone, this increases the fault tolerance of your applications. For example, if one or more target groups does not have a healthy target in an Availability Zone, we remove the IP address for the corresponding subnet from DNS, but the load balancer nodes in the other Availability Zones are still available to route traffic. If a client doesn't honor the time-to-live (TTL) and sends requests to the IP address after it is removed from DNS, the requests fail.

For TCP traffic, the load balancer selects a target using a flow hash algorithm based on the protocol, source IP address, source port, destination IP address, destination port, and TCP sequence number. The TCP connections from a client have different source ports and sequence numbers, and can be routed to different targets. Each individual TCP connection is routed to a single target for the life of the connection.

For UDP traffic, the load balancer selects a target using a flow hash algorithm based on the protocol, source IP address, source port, destination IP address, and destination port. A UDP flow has the same source and destination, so it is consistently routed to a single target throughout its lifetime. Different UDP flows have different source IP addresses and ports, so they can be routed to different targets.

Elastic Load Balancing creates a network interface for each Availability Zone you enable. Each load balancer node in the Availability Zone uses this network interface to get a static IP address. When you create an Internet-facing load balancer, you can optionally associate one Elastic IP address per subnet.

When you create a target group, you specify its target type, which determines whether you register targets by instance ID or IP address. If you register targets by instance ID, the source IP addresses of the clients are preserved and provided to your applications. If you register targets by IP address, the source IP addresses are the private IP addresses of the load balancer nodes.

You can add and remove targets from your load balancer as your needs change, without disrupting the overall flow of requests to your application. Elastic Load Balancing scales your load balancer as traffic to your application changes over time. Elastic Load Balancing can scale to the vast majority of workloads automatically.

You can configure health checks, which are used to monitor the health of the registered targets so that the load balancer can send requests only to the healthy targets.

via - <https://docs.aws.amazon.com/elasticloadbalancing/latest/network/introduction.html>

Incorrect options:

Application Load Balancer - An Application Load Balancer functions at the application layer, the seventh layer of the Open Systems Interconnection (OSI) model. After the load balancer receives a request, it evaluates the listener rules in priority order to determine which rule to apply and then selects a target from the target group for the rule action.

One of many benefits of the Application Load Balancer is its support for path-based routing. You can configure rules for your listener that forward requests based on the URL in the request. This enables you to structure your application as smaller services, and route requests to the correct service based on the content of the URL. For needs relating to network traffic go with Network Load Balancer.

Elastic Load Balancer - Elastic Load Balancing is the service itself that offers different types of load balancers.

Classic Load Balancer - It is a basic load balancer that distributes traffic. If your account was created before 2013-12-04, your account supports EC2-Classic instances and you will benefit in using this type of load balancer. The classic load balancer can be used regardless of when your account was created and whether you use EC2-Classic or whether your instances are in a VPC but just remember its the basic load balancer AWS offers and not advanced as the others.

Question 42

A developer is configuring an Application Load Balancer (ALB) to direct traffic to the application's EC2 instances and Lambda functions. Which of the following characteristics of the ALB can be identified as correct? (Select two)

- If you specify targets using an instance ID, traffic is routed to instances using any private IP address from one or more network interfaces
- If you specify targets using IP addresses, traffic is routed to instances using the primary private IP address
- An ALB has three possible target types: Instance, IP and Lambda
- You can not specify publicly routable IP addresses to an ALB
- An ALB has three possible target types: Hostname, IP and Lambda

Overall explanation

Correct options:

An ALB has three possible target types: Instance, IP and Lambda

When you create a target group, you specify its target type, which determines the type of target you specify when registering targets with this target group. After you create a target group, you cannot change its target type. The following are the possible target types:

12. Instance - The targets are specified by instance ID
13. IP - The targets are IP addresses
14. Lambda - The target is a Lambda function

You can not specify publicly routable IP addresses to an ALB

When the target type is IP, you can specify IP addresses from specific CIDR blocks only. You can't specify publicly routable IP addresses.

Incorrect options:

If you specify targets using an instance ID, traffic is routed to instances using any private IP address from one or more network interfaces - If you specify targets using an instance ID, traffic is routed to instances using the primary private IP address specified in the primary network interface for the instance.

If you specify targets using IP addresses, traffic is routed to instances using the primary private IP address - If you specify targets using IP addresses, you can route traffic to an instance using any private IP address from one or more network interfaces. This enables multiple applications on an instance to use the same port.

An ALB has three possible target types: Hostname, IP and Lambda - This is incorrect, as described in the correct explanation above.

Question 43

A financial services company with over 10,000 employees has hired you as the new Senior Developer. Initially caching was enabled to reduce the number of calls made to all API endpoints and improve the latency of requests to the company's API Gateway.

For testing purposes, you would like to invalidate caching for the API clients to get the most recent responses. Which of the following should you do?

- Using the request parameter ?cache-control-max-age=0
- Using the Header Cache-Control: max-age=0
- Using the Header Bypass-Cache=1
- Use the Request parameter: ?bypass_cache=1

Overall explanation

Correct option:

Using the Header Cache-Control: max-age=0

A client of your API can invalidate an existing cache entry and reload it from the integration endpoint for individual requests. The client must send a request that contains the Cache-Control: max-age=0 header. The client receives the response directly from the integration endpoint instead of the cache, provided that the client is authorized to do so. This replaces the existing cache entry with the new response, which is fetched from the integration endpoint.

Invalidate an API Gateway cache entry

A client of your API can invalidate an existing cache entry and reload it from the integration endpoint for individual requests. The client must send a request that contains the Cache-Control: max-age=0 header. The client receives the response directly from the integration endpoint instead of the cache, provided that the client is authorized to do so. This replaces the existing cache entry with the new response, which is fetched from the integration endpoint.

To grant permission for a client, attach a policy of the following format to an IAM execution role for the user:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "execute-api:Invoke",
        "execute-api:GetMethodCache"
      ],
      "Resource": [
        "arn:aws:execute-api:region:account-id:api-id:stage-name/GET/resource-path-specifier*"
      ]
    }
  ]
}
```

This policy allows the API Gateway execution service to invalidate the cache for requests on the specified resource (or resources). To specify a group of targeted resources, use a wildcard (*) character for account-id, api-id, and other entries in the ARN value of Resource. For more information on how to set permissions for the API Gateway execution service, see [Control access to an API with IAM permissions](#).

If you don't impose an InvokeMethodCache policy (or choose the **Require authorization** check box in the console), any client can invalidate the API cache. If most or all of the clients invalidate the API cache, this could significantly increase the latency of your API.

When the policy is in place, caching is enabled and authorization is required. You can control how unauthorized requests are handled by choosing an option from Handle unauthorized requests in the API Gateway console.

via - <https://docs.aws.amazon.com/apigateway/latest/developerguide/api-gateway-caching.html#invalidate-method-caching>

Incorrect options:

Use the Request parameter: ?bypass_cache=1 - Method parameters take query string but this is not one of them.

Using the Header Bypass-Cache=1 - This is a made-up option.

Using the request parameter ?cache-control-max-age=0 - To invalidate cache it requires a header and not a request parameter.

Question 44

DevOps engineers are developing an order processing system where notifications are sent to a department whenever an order is placed for a product. The system also pushes identical notifications of the new order to a processing module that would allow EC2 instances to handle the fulfillment of the order. In the case of processing errors, the messages should be allowed to be re-processed at a later stage. The order processing system should be able to scale transparently without the need for any manual or programmatic provisioning of resources.

Which of the following solutions can be used to address this use-case in the most cost-efficient way?

- SNS + Lambda
- SNS + SQS
- SQS + SES
- SNS + Kinesis

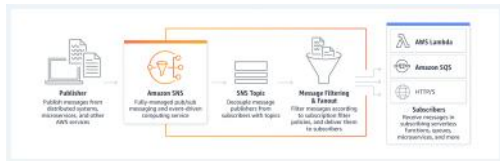
Overall explanation

Correct option:

SNS + SQS

Amazon SNS enables message filtering and fanout to a large number of subscribers, including serverless functions, queues, and distributed systems. Additionally, Amazon SNS fans out notifications to end users via mobile push messages, SMS, and email.

How SNS Works:



via - <https://aws.amazon.com/sns/>

Amazon Simple Queue Service (SQS) is a fully managed message queuing service that enables you to decouple and scale microservices, distributed systems, and serverless applications. SQS offers two types of message queues. Standard queues offer maximum throughput, best-effort ordering, and at-least-once delivery. SQS FIFO queues are designed to guarantee that messages are processed exactly once, in the exact order that they are sent.

Because each buffered request can be processed independently, Amazon SQS can scale transparently to handle the load without any provisioning instructions from you.

SNS and SQS can be used to create a fanout messaging scenario in which messages are "pushed" to multiple subscribers, which eliminates the need to periodically check or poll for updates and enables parallel asynchronous processing of the message by the subscribers. SQS can allow for later re-processing and dead letter queues. This is called the fan-out pattern.

Incorrect options:

SNS + Kinesis - You can use Amazon Kinesis Data Streams to collect and process large streams of data records in real-time. Kinesis Data Streams stores records from 24 hours (by default) to 8760 hours (365 days). However, you need to manually provision shards in case the load increases or you need to use CloudWatch alarms to set up auto scaling for the shards. Since Kinesis only supports transparent scaling in the on-demand mode, however, it is not cost efficient for the given use case, so this option is not the right fit for the given use case.

SNS + Lambda - Amazon SNS and AWS Lambda are integrated so you can invoke Lambda functions with Amazon SNS notifications. The Lambda function receives the message payload as an input parameter and can manipulate the information in the message, publish the message to other SNS topics, or send the message to other AWS services. However, your EC2 instances cannot "poll" from Lambda functions and as such, this would not work.

SQS + SES - This will not work as the messages need to be processed twice (once for sending the notification and later for order fulfillment) and SQS only allows for one consuming application.

Question 45

You have configured a Network ACL and a Security Group for the load balancer and Amazon EC2 instances to allow inbound traffic on port 80. However, users are still unable to connect to your website after launch.

Which additional configuration is required to make the website accessible to all users over the internet?

- Add a rule to the Network ACLs to allow outbound traffic on ports 1025 - 5000
- Add a rule to the Network ACLs to allow outbound traffic on ports 1024 - 65535
- Add a rule to the Security Group allowing outbound traffic on port 80
- Add a rule to the Network ACLs to allow outbound traffic on ports 32768 - 61000

Overall explanation

Correct option:

Add a rule to the Network ACLs to allow outbound traffic on ports 1024 - 65535

A Network Access Control List (ACL) is an optional layer of security for your VPC that acts as a firewall for controlling traffic in and out of one or more subnets. You might set up network ACLs with rules similar to your security groups in order to add an additional layer of security to your VPC.

When you create a custom Network ACL and associate it with a subnet, by default, this custom Network ACL denies all inbound and outbound traffic until you add rules. A network ACL has separate inbound and outbound rules, and each rule can either allow or deny traffic. Network ACLs are stateless, which means that responses to allowed inbound traffic are subject to the rules for outbound traffic (and vice versa).

The client that initiates the request chooses the ephemeral port range. The range varies depending on the client's operating system. Requests originating from Elastic Load Balancing use ports 1024-65535. List of ephemeral port ranges:

15. Many Linux kernels (including the Amazon Linux kernel) use ports 32768-61000.
16. Requests originating from Elastic Load Balancing use ports 1024-65535.
17. Windows operating systems through Windows Server 2003 use ports 1025-5000.
18. Windows Server 2008 and later versions use ports 49152-65535.
19. A NAT gateway uses ports 1024-65535.

Incorrect options:

Add a rule to the Network ACLs to allow outbound traffic on ports 1025 - 5000 - As discussed above, Windows operating systems through Windows Server 2003 use ports 1025-5000. ELB uses the port range 1024-65535.

Add a rule to the Network ACLs to allow outbound traffic on ports 32768 - 61000 - As discussed above, Linux kernels (including the Amazon Linux kernel) use ports 1025-5000. ELB uses the port range 1024-65535.

Add a rule to the Security Group allowing outbound traffic on port 80 - A security group acts as a virtual firewall for your instance to control inbound and outbound traffic. Security groups act at the instance level, not the subnet level. Security groups are stateful — if you send a request from your instance, the response traffic for that request is allowed to flow in regardless of inbound security group rules.

Responses to allowed inbound traffic are allowed to flow out, regardless of outbound rules.

References:

<https://docs.aws.amazon.com/vpc/latest/userguide/vpc-network-acls.html>

https://docs.aws.amazon.com/vpc/latest/userguide/VPC_SecurityGroups.html

Question 46

A financial services company has developed a REST API which is deployed in an Auto Scaling Group behind an Application Load Balancer. The API stores the data payload in DynamoDB and the static content is served through S3. Upon analyzing the usage pattern, it's found that 80% of the read requests are shared across all users.

As a Developer Associate, how can you improve the application performance while optimizing the cost with the least development effort?

- Enable ElastiCache Redis for DynamoDB and ElastiCache Memcached for S3
- Enable DAX for DynamoDB and ElastiCache Memcached for S3
- Enable ElastiCache Redis for DynamoDB and CloudFront for S3
- Enable DynamoDB Accelerator (DAX) for DynamoDB and CloudFront for S3

Overall explanation

Correct option:

Enable DynamoDB Accelerator (DAX) for DynamoDB and CloudFront for S3

DynamoDB Accelerator (DAX) is a fully managed, highly available, in-memory cache for Amazon DynamoDB that delivers up to a 10 times performance improvement—from milliseconds to microseconds—even at millions of requests per second.

DAX is tightly integrated with DynamoDB—you simply provision a DAX cluster, use the DAX client SDK to point your existing DynamoDB API calls at the DAX cluster, and let DAX handle the rest. Because DAX is API-compatible with DynamoDB, you don't have to make any functional application code changes. DAX is used to natively cache DynamoDB reads.

CloudFront is a content delivery network (CDN) service that delivers static and dynamic web content, video streams, and APIs around the world, securely and at scale. By design, delivering data out of CloudFront can be more cost-effective than delivering it from S3 directly to your users.

When a user requests content that you serve with CloudFront, their request is routed to a nearby Edge Location. If CloudFront has a cached copy of the requested file, CloudFront delivers it to the user, providing a fast (low-latency) response. If the file they've requested isn't yet cached, CloudFront retrieves it from your origin — for example, the S3 bucket where you've stored your content.

So, you can use CloudFront to improve application performance to serve static content from S3.

Incorrect options:

Enable ElastiCache Redis for DynamoDB and CloudFront for S3

Amazon ElastiCache for Redis is a blazing fast in-memory data store that provides sub-millisecond latency to power internet-scale real-time applications. Amazon ElastiCache for Redis is a great choice for real-time transactional and analytical processing use cases such as caching, chat/messaging, gaming leaderboards, geospatial, machine learning, media streaming, queues, real-time analytics, and session store.

ElastiCache for Redis Overview:



via - <https://aws.amazon.com/elasticache/redis/>

Although, you can integrate Redis with DynamoDB, it's much more involved from a development perspective. For the given use-case, you should use DAX which is a much better fit.

Enable DAX for DynamoDB and ElastiCache Memcached for S3

Enable ElastiCache Redis for DynamoDB and ElastiCache Memcached for S3

Amazon ElastiCache for Memcached is a Memcached-compatible in-memory key-value store service that can be used as a cache or a data store. Amazon ElastiCache for Memcached is a great choice for implementing an in-memory cache to decrease access latency, increase throughput, and ease the load off your relational or NoSQL database.

ElastiCache cannot be used as a cache to serve static content from S3, so both these options are incorrect.

Question 47

A media analytics company has built a streaming application on Lambda using Serverless Application Model (SAM).

As a Developer Associate, which of the following would you identify as the correct order of execution to successfully deploy the application?

- Develop the SAM template locally => upload the template to CodeCommit => deploy your application to CodeDeploy
- Develop the SAM template locally => upload the template to S3 => deploy your application to the cloud
- Develop the SAM template locally => deploy the template to S3 => use your application in the cloud
- Develop the SAM template locally => upload the template to Lambda => deploy your application to the cloud

Overall explanation

Correct option:

Develop the SAM template locally => upload the template to S3 => deploy your application to the cloud

The AWS Serverless Application Model (SAM) is an open-source framework for building serverless applications. It provides shorthand syntax to express functions, APIs, databases, and event source mappings. With just a few lines per resource, you can define the application you want and model it using YAML.

You can develop and test your serverless application locally, and then you can deploy your application by using the `sam deploy` command. The `sam deploy` command zips your application artifacts, uploads them to Amazon Simple Storage Service (Amazon S3), and deploys your application to the AWS Cloud. AWS SAM uses AWS CloudFormation as the underlying deployment mechanism.

Deploying Serverless Applications

[View](#) | [Watch](#) | [Edit](#)

AWS SAM uses AWS CloudFormation as the underlying deployment mechanism. For more information, see [What Is AWS CloudFormation?](#).

You can deploy your application by using AWS SAM command line interface (CLI) commands. You can also use other AWS services that integrate with AWS SAM to automate your deployments.

The standard input to deploying serverless applications is the build artifacts created using the `sam build` command. For more information about the `sam build` command, see [Building Serverless Applications](#).

Deploying Using the AWS SAM CLI

After you develop and test your serverless application locally, you can deploy your application by using the `sam deploy` command.

If you want AWS SAM to guide you through the deployment with prompts, specify the `--guided` flag. When you specify this flag, the `sam deploy` command zip your application artifacts, uploads them to Amazon Simple Storage Service (Amazon S3), and deploys your application to the AWS Cloud.

Example:

```
# Deploy an application using prompts:
sam deploy --guided
```

Publishing Serverless Applications

The AWS Serverless Application Repository is a service that hosts serverless applications that are built using AWS SAM. If you want to share serverless applications with others, you can publish them in the AWS Serverless Application Repository. You can also search, browse, and deploy serverless applications that have been published by others. For more information, see [What Is the AWS Serverless Application Repository?](#).

Automating Deployments

You can use AWS SAM with a number of other AWS services to automate the deployment process of your serverless application.

- **CodeBuild:** You use CodeBuild to build, locally test, and package your serverless application. For more information, see [What Is CodeBuild?](#).
- **CodeDeploy:** You use CodeDeploy to gradually deploy updates to your serverless applications. For more information, see [Deploying Serverless Applications Gradually](#).
- **CodePipeline:** You use CodePipeline to model, visualize, and automate the steps that are required to release your serverless application. For more information, see [What Is CodePipeline?](#).

via - <https://docs.aws.amazon.com/serverless-application-model/latest/developerguide/serverless-deploying.html>

Incorrect options:

Develop the SAM template locally => upload the template to Lambda => deploy your application to the cloud

Develop the SAM template locally => upload the template to S3 => deploy your application to CodeDeploy

Develop the SAM template locally => deploy the template to S3 => use your application in the cloud

These three options contradict the details provided in the explanation above, so these are incorrect.

Question 48

A developer has created a new Application Load Balancer but has not registered any targets with the target groups.

Which of the following errors would be generated by the Load Balancer?

- HTTP 500: Internal server error
- HTTP 503: Service unavailable
- HTTP 502: Bad gateway
- HTTP 504: Gateway timeout

Overall explanation

Correct option:

HTTP 503: Service unavailable

The Load Balancer generates the HTTP 503: Service unavailable error when the target groups for the load balancer have no registered targets.

Incorrect options:

HTTP 500: Internal server error

HTTP 502: Bad gateway

HTTP 504: Gateway timeout

Here is a summary of the possible causes for these error types:

HTTP 500: Internal server error

Possible causes:

- You configured an AWS IAM user with access control (or lack of) and there was an error accessing the web ACL rules.
- The load balancer is unable to communicate with the IP address endpoint or the IP user info endpoint. Verify that the security groups for your load balancer and the network ACLs for your VPC allow inbound access to these endpoints. Verify that your VPC has internet access. If you have an internet-facing load balancer, use a NAT gateway to enable internet access.

HTTP 501: Not implemented

The load balancer received a `Transfer-Encoding` header with an unsupported value. The supported values for `Transfer-Encoding` are `Chunked` and `Deflate`. As an alternative, you can use the `Content-Encoding` header.

HTTP 502: Bad gateway

Possible causes:

- The load balancer received a TCP RST from the target when attempting to establish a connection.
- The load balancer received an unexpected response from the target, such as "TCP Distribution unavailable (that connection)", when attempting to establish a connection. Check whether traffic is allowed from the load balancer address to the target on the target port.
- The target closed the connection with a TCP RST or a TCP FIN while the load balancer had an outstanding request to the target. Check whether the keep-alive duration of the target is shorter than the idle timeout value of the load balancer.
- The target response is malformed or contains HTTP headers that are not valid.
- The load balancer encountered an SSL handshake error or SSL handshake timeout (10 seconds) when connecting to a target.
- The target response delay is not supported for a request being handled by a target that was designated. Increase the delay period so that simplify operations can complete.
- The target is a Lambda function and the response body exceeds 1 MB.
- The target is a Lambda function that did not respond before its configured timeout was reached.

HTTP 503: Service unavailable

The target groups for the load balancer have no registered targets.

HTTP 504: Gateway timeout

Possible causes:

- The load balancer failed to establish a connection to the target before the connection timeout expired (10 seconds).
- The load balancer established a connection to the target but the target did not respond before the idle timeout period elapsed.
- The network ACL for the subnet did not allow traffic from the targets to the load balancer nodes on the optional ports (5024-50325).
- The target returns a connection-length greater than 1 is larger than the waiting time. The load balancer timed out waiting for the response bytes.
- The target is a Lambda function and the Lambda service did not respond before the connection timeout was reached.

via - <https://docs.aws.amazon.com/elasticloadbalancing/latest/application/load-balancer-troubleshooting.html>

Question 49

You are using AWS SQS FIFO queues to get the ordering of messages on a per `user_id` basis.

As a developer, which message parameter should you set the value of user_id to guarantee the ordering?

- MessageDeduplicationId
- MessageHash
- MessageGroupId
- MessageOrderId

Overall explanation

Correct option:

AWS FIFO queues are designed to enhance messaging between applications when the order of operations and events has to be enforced.

FIFO (First-in-First-Out) queues are designed to enhance messaging between applications when the order of operations and events is critical, or where duplicates can't be tolerated, for example:

- Ensure that user-entered commands are executed in the right order.
- Display the correct product price by sending price modifications in the right order.
- Prevent a student from enrolling in a course before registering for an account.

FIFO queues also provide exactly-once processing but have a limited number of transactions per second (TPS):

- If you use batching, FIFO queues support up to 3,000 transactions per second, per API method (SendMessageBatch, ReceiveMessage, or DeleteMessageBatch). The 3000 transactions represent 300 API calls, each with a batch of 10 messages. To request a quota increase, submit a support request [link](#).
- Without batching, FIFO queues support up to 300 API calls per second, per API method (SendMessage, ReceiveMessage, or DeleteMessage).

Note

- Amazon SNS isn't currently compatible with FIFO queues.
- The name of a FIFO queue must end with the .fifo suffix. The suffix counts towards the 80-character queue name quota. To determine whether a queue is FIFO, you can check whether the queue name ends with the suffix.

via - <https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/FIFO-queues.html>

MessageGroupId

The message group ID is the tag that specifies that a message belongs to a specific message group. Messages that belong to the same message group are always processed one by one, in a strict order relative to the message group (however, messages that belong to different message groups might be processed out of order).

Incorrect options:

MessageDeduplicationId - The message deduplication ID is the token used for the deduplication of sent messages. If a message with a particular message deduplication ID is sent successfully, any messages sent with the same message deduplication ID are accepted successfully but aren't delivered during the 5-minute deduplication interval.

MessageOrderId - This is a made-up option and has been added as a distractor.

MessageHash - This is a made-up option and has been added as a distractor.

Question 50

You would like your Elastic Beanstalk environment to expose an HTTPS endpoint instead of an HTTP endpoint to get in-flight encryption between your clients and your web servers. What must be done to set up HTTPS on Beanstalk?

- Create a config file in the .ebextensions folder to configure the Load Balancer
- Use a separate CloudFormation template to load the SSL certificate onto the Load Balancer
- Configure Health Checks
- Open up the port 80 for the security group

Overall explanation

Correct option:

The simplest way to use HTTPS with an Elastic Beanstalk environment is to assign a server certificate to your environment's load balancer. When you configure your load balancer to terminate HTTPS, the connection between the client and the load balancer is secure. Backend connections between the load balancer and EC2 instances use HTTP, so no additional configuration of the instances is required.

Configuring HTTPS for your Elastic Beanstalk environment

for - Beanstalk

If you've purchased and configured a custom domain name for your Elastic Beanstalk environment, you can use HTTPS to allow users to connect to your web site securely. If you don't own a domain name, you can still use HTTPS with a self-signed certificate for development and testing purposes. HTTPS is a must for any application that handles user data or login information.

The simplest way to use HTTPS with an Elastic Beanstalk environment is to assign a server certificate to your environment's load balancer. When you configure your load balancer to terminate HTTPS, the connection between the client and the load balancer is secure. Backend connections between the load balancer and EC2 instances use HTTP, so no additional configuration of the instances is required.

Note

- With AWS Certificate Manager (ACM), you can create a trusted certificate for your domain names for free. ACM certificates can only be used with AWS load balancers and Amazon CloudFront distributions, and ACM is available only in certain AWS Regions.

To use an ACM certificate with Elastic Beanstalk, see [Configuring your Elastic Beanstalk environment's load balancer to terminate HTTPS](#).

If you run your application in a single instance environment, or need to secure the connection all the way to the EC2 instances behind the load balancer, you can configure the proxy server that runs on the instance to terminate HTTPS. Configuring your instance to terminate HTTPS connections requires the use of configuration files to modify the software running on the instance, and to modify security groups to allow secure connections.

To end-to-end HTTPS in a load balancer environment, you can configure instance and load balancer termination to encrypt both connections. By default, if you configure the load balancer to forward traffic using HTTPS, it will trust any certificate presented to it by the backend instances. For maximum security, you can attach policies to the load balancer that prevent it from connecting to instances that don't present a public certificate that is trusted.

Note

- You can also configure the load balancer to relay HTTPS traffic without decrypting it. The down side to this method is that the load balancer cannot see the requests and that, current options routing or request response metrics.

If ACM is not available in your region, you can purchase a trusted certificate from a third party. A third-party certificate can be used to encrypt HTTPS traffic to your load balancer, on the backend instances, or both.

For development and testing, you can create and sign a certificate yourself with open-source tools. Self-signed certificates are free and easy to create, but cannot be used for front-end decryption or public sites. If you attempt to use a self-signed certificate for an HTTPS connection to a client, the client's browser displays an error message indicating that your web site is unsafe. You can, however, use a self-signed certificate for development and testing purposes.

ACM is the preferred tool to provision, manage, and deploy your server certificates programmatically or using the AWS CLI. If ACM is not available in your AWS Region, you can upload a third-party or self-signed certificate and provide key to AWS Identity and Access Management (IAM) by using the AWS CLI. Certificates stored in IAM can be used with load balancers and CloudFront distributions.

via - <https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/configuring-https.html>

Create a config file in the .ebextensions folder to configure the Load Balancer

To update your AWS Elastic Beanstalk environment to use HTTPS, you need to configure an HTTPS listener for the load balancer in your environment. Two types of load balancers support an HTTPS listener: Classic Load Balancer and Application Load Balancer.

Example .ebextensions/securelistener-alb.config

Use this example when your environment has an Application Load Balancer. The example uses options in the aws:elbv2:listener namespace to configure an HTTPS listener on port 443 with the specified certificate. The listener routes traffic to the default process.

option_settings:

```
aws:elbv2:listener:443:
  ListenerEnabled: 'true'
  Protocol: HTTPS
  SSLCertificateArns: arn:aws:acm:us-east-2:123456789012:certificate/#####
```

Incorrect options:

Use a separate CloudFormation template to load the SSL certificate onto the Load Balancer - A separate CloudFormation template won't be able to mutate the state of a Load Balancer managed by Elastic Beanstalk, so this option is incorrect.

Open up the port 80 for the security group - Port 80 is for HTTP traffic, so this option is incorrect.

Configure Health Checks - Health Checks are not related to SSL certificates, so this option is ruled out.

References:

<https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/configuring-https-elb.html>

<https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/configuring-https.html>

Question 51

Your Lambda function must use the Node.js drivers to connect to your RDS PostgreSQL database in your VPC.

How do you bundle your Lambda function to add the dependencies?

- Zip the function and the dependencies separately and upload them in AWS Lambda as two parts
- Upload the code through the AWS console and upload the dependencies as a zip
- Zip the function as-is with a package.json file so that AWS Lambda can resolve the dependencies for you
- Put the function and the dependencies in one folder and zip them together

Overall explanation

Correct option:

AWS Lambda lets you run code without provisioning or managing servers. You pay only for the compute time you consume.

How Lambda function works:



via - <https://aws.amazon.com/lambda/>

Put the function and the dependencies in one folder and zip them together

A deployment package is a ZIP archive that contains your function code and dependencies. You need to create a deployment package if you use the Lambda API to manage functions, or if you need to include libraries and dependencies other than the AWS SDK. You can upload the package directly to Lambda, or you can use an Amazon S3 bucket, and then upload it to Lambda. If the deployment package is larger than 50 MB, you must use Amazon S3. This is the standard way of packaging Lambda functions.

Incorrect options:

Zip the function as-is with a package.json file so that AWS Lambda can resolve the dependencies for you

Upload the code through the AWS console and upload the dependencies as a zip

Zip the function and the dependencies separately and upload them in AWS Lambda as two parts

These three options are incorrect as there's only one way of deploying a Lambda function, which is to provide the zip file with all dependencies that it'll need.

Question 52

Your company wants to move away from manually managing Lambda in the AWS console and wants to upload and update them using AWS CloudFormation.

How do you declare an AWS Lambda function in CloudFormation? (Select two)

- Upload all the code to CodeCommit and refer to the CodeCommit Repository in AWS::Lambda::Function block
- Upload all the code as a folder to S3 and refer the folder in AWS::Lambda::Function block
- Write the AWS Lambda code inline in CloudFormation in the AWS::Lambda::Function block and reference the dependencies as a zip file stored in S3
- Write the AWS Lambda code inline in CloudFormation in the AWS::Lambda::Function block as long as there are no third-party dependencies
- Upload all the code as a zip to S3 and refer the object in AWS::Lambda::Function block

Overall explanation

Correct options:

AWS Lambda lets you run code without provisioning or managing servers. You pay only for the compute time you consume.

How Lambda function works:



via - <https://aws.amazon.com/lambda/>

Upload all the code as a zip to S3 and refer the object in AWS::Lambda::Function block

You can upload all the code as a zip to S3 and refer the object in AWS::Lambda::Function block.

The AWS::Lambda::Function resource creates a Lambda function. To create a function, you need a deployment package and an execution role. The deployment package contains your function code.

Write the AWS Lambda code inline in CloudFormation in the AWS::Lambda::Function block as long as there are no third-party dependencies

The other option is to write the code inline for Node.js and Python as long as there are no dependencies for your code, besides the dependencies already provided by AWS in your Lambda Runtime (aws-sdk and cf-response and many other AWS related libraries are preloaded via, for example, boto3 (python) in the lambda instances.)

YAML template for creating a Lambda function:

Type: AWS::Lambda::Function

Properties:

```
Code:
  Code
DeadLetterConfig:
  DeadLetterConfig
Description: String
Environment:
  Environment
FileSystemConfigs:
  - FileSystemConfig
FunctionName: String
Handler: String
KmsKeyArn: String
Layers:
  - String
MemorySize: Integer
ReservedConcurrentExecutions: Integer
Role: String
Runtime: String
Tags:
  - Tag
Timeout: Integer
TracingConfig:
  TracingConfig
VpcConfig:
  VpcConfig
```

Incorrect options:

Upload all the code to CodeCommit and refer to the CodeCommit Repository in AWS::Lambda::Function block

Upload all the code as a folder to S3 and refer the folder in AWS::Lambda::Function block

Write the AWS Lambda code inline in CloudFormation in the AWS::Lambda::Function block and reference the dependencies as a zip file stored in S3

These three options contradict the explanation provided earlier. So these are incorrect.

Question 53

An IT company uses AWS CloudFormation templates to provision their AWS infrastructure for Amazon EC2, Amazon VPC, and Amazon S3 resources. Using cross-stack referencing, a developer creates a stack called NetworkStack which will export the subnetId that can be used when creating EC2 instances in another stack.

To use the exported value in another stack, which of the following functions must be used?

- !Sub
- !Ref
- !ImportValue
- !GetAtt

Overall explanation

Correct option:

!ImportValue

The intrinsic function Fn::ImportValue returns the value of an output exported by another stack. You typically use this function to create cross-stack references.

Incorrect options:

!Ref - Returns the value of the specified parameter or resource.

!GetAtt - Returns the value of an attribute from a resource in the template.

!Sub - Substitutes variables in an input string with values that you specify.

Reference:

<https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/intrinsic-function-reference-importvalue.html>

Question 54

A popular mobile app retrieves data from an AWS DynamoDB table that was provisioned with read-capacity units (RCU's) that are evenly shared across four partitions. One of those partitions is receiving more traffic than the other partitions, causing hot partition issues. What technology will allow you to reduce the read traffic on your AWS DynamoDB table with minimal effort?

- DynamoDB DAX
- DynamoDB Streams
- More partitions
- ElastiCache

Overall explanation
Correct option:

DynamoDB DAX

Amazon DynamoDB Accelerator (DAX) is a fully managed, highly available, in-memory cache for DynamoDB that delivers up to a 10x performance improvement: from milliseconds to microseconds: even at millions of requests per second.

Incorrect options:

DynamoDB Streams - A stream record contains information about a data modification to a single item in a DynamoDB table. This is not the correct option for the given use-case.

ElastiCache - ElastiCache can cache the results from anything but you will need to modify your code to check the cache before querying the main query store. As the given use-case mandates minimal effort, so this option is not correct.

More partitions - This option has been added as a distractor as DynamoDB handles that for you automatically.

Question 55

A business-critical mobile application uses Amazon Cognito user pools with multi-factor authentication (MFA) enabled for all its users. The application manages confidential data about the company's sales forecasts and product launches. Considering the highly critical nature of the application, the company wants to track every user login activity via a notification sent as an email to the security team. Which of the following would you recommend as the MOST optimal way of implementing this requirement within a short period?

- Create an AWS Lambda function that uses Amazon Simple Email Service to send an email notification to the concerned security team. Configure this function as Amazon Cognito post-authentication Lambda trigger
- Configure Amazon Cognito user pools authenticated API operations and MFA API operations to send all login data to Amazon Kinesis Data Streams. Configure an AWS Lambda function to analyze these streams and trigger an SNS notification to the security team based on user access
- Configure an AWS Lambda function as a trigger to Amazon Cognito identity pools authenticated API operations. Create the Lambda function to utilize the Amazon Simple Email Service to send an email notification to the concerned security team
- Create an AWS Lambda function that uses Amazon Simple Email Service to send an email notification to the concerned security team. Configure this function as Amazon Cognito pre-authentication Lambda trigger

Overall explanation

Correct option:

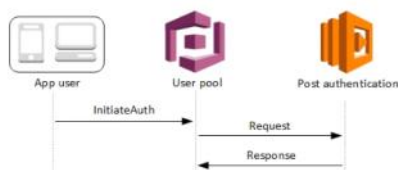
Create an AWS Lambda function that uses Amazon Simple Email Service to send an email notification to the concerned security team. Configure this function as Amazon Cognito post-authentication Lambda trigger

Amazon Cognito invokes Post authentication Lambda trigger after signing in a user, you can add custom logic after Amazon Cognito authenticates the user.

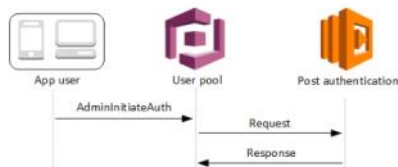
Post-authentication Lambda flows:

Post authentication Lambda flows

Client authentication flow



Server authentication flow



via - <https://docs.aws.amazon.com/cognito/latest/developerguide/user-pool-lambda-post-authentication.html>

Incorrect options:

Create an AWS Lambda function that uses Amazon Simple Email Service to send an email notification to the concerned security team. Configure this function as Amazon Cognito pre-authentication Lambda trigger - Pre-authentication Lambda trigger: Amazon Cognito invokes this trigger when a user attempts to sign in so that you can create custom validation that accepts or denies the authentication request. This is not useful for the current use case since we want to track user login activity which happens post-authentication.

Configure an AWS Lambda function as a trigger to Amazon Cognito identity pools authenticated API operations. Create the Lambda function to utilize the Amazon Simple Email Service to send an email notification to the concerned security team - This statement is incorrect. Amazon Cognito identity pools (federated identities) enable you to create unique identities for your users and federate them with identity providers. Cognito identity pools are for authorization and not for authentication.

Configure Amazon Cognito user pools authenticated API operations and MFA API operations to send all login data to Amazon Kinesis Data Streams. Configure an AWS Lambda function to analyze these streams and trigger an SNS notification to the security team based on user access - This is a made-up option given only as a distractor.

Question 56

A company's e-commerce application becomes slow when traffic spikes. The application has a three-tier architecture (web, application and database tier) that uses synchronous transactions. The development team at the company has identified certain bottlenecks in the application tier and it is looking for a long term solution to improve the application's performance. As a developer associate, which of the following solutions would you suggest to meet the required application response times while accounting for any traffic spikes?

- Leverage SQS with asynchronous AWS Lambda calls to decouple the application and data tiers
- Leverage horizontal scaling for the application's persistence layer by adding Oracle RAC on AWS
- Leverage horizontal scaling for the web and application tiers by using Auto Scaling groups and Application Load Balancer
- Leverage vertical scaling for the application instance by provisioning a larger Amazon EC2 instance size

Overall explanation

Correct option:

Leverage horizontal scaling for the web and application tiers by using Auto Scaling groups and Application Load Balancer - A horizontally scalable system is one that can increase capacity by adding more computers to the system. This is in contrast to a vertically scalable system, which is constrained to running its processes on only one computer; in such systems, the only way to increase performance is to add more resources into one computer in the form of faster (or more) CPUs, memory or storage.

Horizontally scalable systems are oftentimes able to outperform vertically scalable systems by enabling parallel execution of workloads and distributing those across many different computers.

Elastic Load Balancing is used to automatically distribute your incoming application traffic across all the EC2 instances that you are running. You can use Elastic Load Balancing to manage incoming requests by optimally routing traffic so that no one instance is overwhelmed.

To use Elastic Load Balancing with your Auto Scaling group, you attach the load balancer to your Auto Scaling group to register the group with the load balancer. Your load balancer acts as a single point of contact for all incoming web traffic to your Auto Scaling group.

When you use Elastic Load Balancing with your Auto Scaling group, it's not necessary to register individual EC2 instances with the load balancer. Instances that are launched by your Auto Scaling group are automatically registered with the load balancer. Likewise, instances that are terminated by your Auto Scaling group are automatically deregistered from the load balancer.

This option will require fewer design changes, it's mostly configuration changes and the ability for the web/application tier to be able to communicate across instances. Hence, this is the right solution for

the current use case.

Incorrect options:

Leverage SQS with asynchronous AWS Lambda calls to decouple the application and data tiers - This is incorrect as it uses asynchronous AWS Lambda calls and the application uses synchronous transactions. The question says there should be no change in the application architecture.

Leverage horizontal scaling for the application's persistence layer by adding Oracle RAC on AWS - The issue is not with the persistence layer at all. This option has only been used as a distractor.

You can deploy scalable Oracle Real Application Clusters (RAC) on Amazon EC2 using Amazon Machine Images (AMI) on AWS Marketplace. Oracle RAC is a shared-everything database cluster technology from Oracle that allows a single database (a set of data files) to be concurrently accessed and served by one or many database server instances.

Leverage vertical scaling for the application instance by provisioning a larger Amazon EC2 instance size - Vertical scaling is just a band-aid solution and will not work long term.

Question 57

A company has hosted its restaurant review website on an Amazon EC2 instance. The website supports multiple languages and the preferred language is added as a query string parameter to the request. The directory structure and file names for all versions of the website are identical. The website responds with the chosen language's webpage when accessed directly. However, when the same webpage is accessed through the configured CloudFront distribution, it defaults to a single language.

How will you fix this issue?

- Create a new cache policy for the CloudFront distribution and set the cache behavior to None to improve caching performance. Update the CloudFront distribution to use the new cache policy
- Create a new cache policy for the CloudFront distribution and set the cache behavior to Query string forwarding and caching. In the Query string whitelist field include the language string. Update the CloudFront distribution to use the new cache policy
- Create a new cache policy for the CloudFront distribution and set the cache behavior to Cache based on selected request headers. Use Whitelist Headers as the caching criteria
- Choose the Customize option for the Object Caching setting and reduce the Default TTL value so that CloudFront forwards requests to your origin more frequently

Overall explanation

Correct option:

Create a new cache policy for the CloudFront distribution and set the cache behavior to Query string forwarding and caching. In the Query string whitelist field include the language string. Update the CloudFront distribution to use the new cache policy

CloudFront can cache different versions of your content based on the values of query string parameters. Forward all, cache based on whitelist option should be chosen if your origin server returns different versions of your objects based on one or more query string parameters. Then specify the parameters that you want CloudFront to use as a basis for caching in the Query string whitelist field.

Incorrect options:

Create a new cache policy for the CloudFront distribution and set the cache behavior to None to improve caching performance. Update the CloudFront distribution to use the new cache policy - None improves Caching. Choose this option if your origin returns the same version of an object regardless of the values of query string parameters. This increases the likelihood that CloudFront can serve a request from the cache, which improves performance and reduces the load on your origin.

Create a new cache policy for the CloudFront distribution and set the cache behavior to Cache based on selected request headers. Use Whitelist Headers as the caching criteria - Cache based on selected request headers is not a valid option since the use case mentions using query string parameters.

Choose the Customize option for the Object Caching setting and reduce the Default TTL value so that CloudFront forwards requests to your origin more frequently - Default TTL specifies the default amount of time, in seconds, that you want objects to stay in CloudFront caches before CloudFront forwards another request to your origin to determine whether the object has been updated. This option is irrelevant for the current use case since the response returned defaulting to the same language is not a TTL issue.

Question 58

You would like to retrieve a subset of your dataset stored in S3 with the CSV format. You would like to retrieve a month of data and only 3 columns out of the 10.

You need to minimize compute and network costs for this, what should you use?

- S3 Inventory
- S3 Select
- S3 Analytics
- S3 Access Logs

Overall explanation

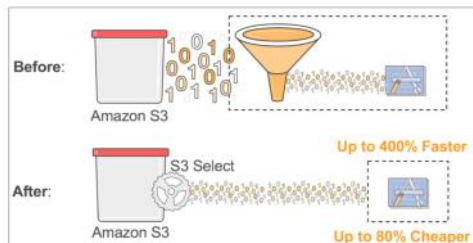
Correct option:

S3 Select

S3 Select enables applications to retrieve only a subset of data from an object by using simple SQL expressions. By using S3 Select to retrieve only the data needed by your application, you can achieve drastic performance increases in many cases you can get as much as a 400% improvement.

S3 Select

S3 Select, launching in preview now generally available, enables applications to retrieve only a subset of data from an object by using simple SQL expressions. By using S3 Select to retrieve only the data needed by your application, you can achieve drastic performance increases - in many cases you can get as much as a 400% improvement.



As an example, let's imagine you're a developer at a large retailer and you need to analyze the weekly sales data from a single store, but the data for all 200 stores is saved in a new GZIP-ed CSV every day. Without S3 Select, you would need to download, decompress and process the entire CSV to get the data you needed. With S3 Select, you can use a simple SQL expression to return only the data from the store you're interested in, instead of retrieving the entire object. This means you're dealing with an order of magnitude less data which improves the performance of your underlying applications.

via - <https://aws.amazon.com/blogs/aws/s3-glacier-select/>

Incorrect options:

S3 Inventory - Amazon S3 inventory is one of the tools Amazon S3 provides to help manage your storage. You can use it to audit and report on the replication and encryption status of your objects for business, compliance, and regulatory needs.

S3 Analytics - By using Amazon S3 analytics storage class analysis you can analyze storage access patterns to help you decide when to transition the right data to the right storage class. This new Amazon S3 analytics feature observes data access patterns to help you determine when to transition less frequently accessed STANDARD storage to the STANDARD_IA (IA, for infrequent access) storage class.

S3 Access Logs - Server access logging provides detailed records for the requests that are made to a bucket. Server access logs are useful for many applications. For example, access log information can be useful in security and access audits. It can also help you learn about your customer base and understand your Amazon S3 bill.

Question 59

Your organization has a single Amazon Simple Storage Service (S3) bucket that contains folders labeled with customer names. Several administrators have IAM access to the S3 bucket and versioning is enabled to easily recover from unintended user actions.

Which of the following statements about versioning is **NOT** true based on this scenario?

- Overwriting a file increases its versions
- Deleting a file is a recoverable operation
- Any file that was unversioned before enabling versioning will have the 'null' version
- Versioning can be enabled only for a specific folder

Overall explanation

Correct option:

Versioning can be enabled only for a specific folder

The versioning state applies to all (never some) of the objects in that bucket. The first time you enable a bucket for versioning, objects in it are thereafter always versioned and given a unique version ID. Versioning Overview:

Versioning is a means of keeping multiple variants of an object in the same bucket. You can use versioning to preserve, retrieve, and restore every version of every object stored in your Amazon S3 bucket. With versioning, you can easily recover from both unintended user actions and application failures. When you enable versioning for a bucket, Amazon S3 receives multiple write requests for the same object simultaneously, it stores all of the objects.

If you enable versioning for a bucket, Amazon S3 automatically generates a unique version ID for the object being stored. In one bucket, for example, you can have two objects with the same key, but different version IDs, such as photo.jpg (version 111111) and photo.jpg (version 121212).



Versioning-enabled buckets enable you to recover objects from accidental deletions or overwrites. For example:

- If you delete an object, instead of removing it permanently, Amazon S3 inserts a delete marker, which becomes the current object version. You can always restore the previous version. For more information, see [Deleting object versions](#).
- If you overwrite an object, it results in a new object version in the bucket. You can always restore the previous version.

Important

If you have an object expiration lifecycle policy in your non-versioned bucket and you want to maintain the same permanent delete behavior when you enable versioning, you must add a noncurrent expiration policy. The noncurrent-expiration lifecycle policy will manage the deletion of the noncurrent object versions in the version-enabled bucket. A version-enabled bucket maintains one current and zero or more noncurrent object versions. For more information, see [How to Create a Lifecycle Policy for an S3 Bucket](#) in the *Amazon Simple Storage Service Console User Guide*.

Buckets can be in one of three states: unversioned (the default), versioning-enabled, or versioning-suspended.

Important

Once you version-enable a bucket, it can never return to an unversioned state. You can, however, suspend versioning on that bucket.

The versioning state applies to all (lower some) of the objects in that bucket. The first time you enable a bucket for versioning, objects in it are thereafter always versioned and given a unique version ID. Note the following:

- **Objects stored in your bucket before you set the versioning state have a version ID of null.** When you enable versioning, existing objects in your bucket do not change. What changes is how Amazon S3 handles the objects in future requests. For more information, see [Handling objects in a versioning-enabled bucket](#).
- The bucket owner (or any user with appropriate permissions) can suspend versioning to stop creating object versions. **When you suspend versioning, existing objects in your bucket do not change.** What changes is how Amazon S3 handles objects in future requests. For more information, see [Handling objects in a versioning-suspended bucket](#).

via - <https://docs.aws.amazon.com/AmazonS3/latest/dev/Versioning.html>

Incorrect options:

Overwriting a file increases its versions - If you overwrite an object (file), it results in a new object version in the bucket. You can always restore the previous version.

Deleting a file is a recoverable operation - Correct, when you delete an object (file), Amazon S3 inserts a delete marker, which becomes the current object version and you can restore the previous version.

Any file that was unversioned before enabling versioning will have the 'null' version - Objects stored in your bucket before you set the versioning state have a version ID of null. Those existing objects in your bucket do not change.

Question 60

The development team at an e-commerce company is preparing for the upcoming Thanksgiving sale. The product manager wants the development team to implement appropriate caching strategy on Amazon ElastiCache to withstand traffic spikes on the website during the sale. A key requirement is to facilitate consistent updates to the product prices and product description, so that the cache never goes out of sync with the backend.

As a Developer Associate, which of the following solutions would you recommend for the given use-case?

- Use a caching strategy to write to the cache directly and sync the backend at a later time
- Use a caching strategy to write to the backend first and wait for the cache to expire via TTL
- Use a caching strategy to write to the backend first and then invalidate the cache
- Use a caching strategy to update the cache and the backend at the same time

Overall explanation

Correct option:

Amazon ElastiCache allows you to seamlessly set up, run, and scale popular open-Source compatible in-memory data stores in the cloud. Build data-intensive apps or boost the performance of your existing databases by retrieving data from high throughput and low latency in-memory data stores. Amazon ElastiCache is a popular choice for real-time use cases like Caching, Session Stores, Gaming, Geospatial Services, Real-Time Analytics, and Queuing.

Broadly, you can set up two types of caching strategies:

20. Lazy Loading
21. Write-Through

Lazy Loading

As the name implies, **lazy loading is a caching strategy that loads data into the cache only when necessary.** It works as described following.

Amazon ElastiCache is an in-memory key-value store that sits between your application and the data store (database) that it accesses. Whenever your application requests data, it first makes the request to the ElastiCache cache. **If the data exists in the cache and is current, ElastiCache returns the data to your application. If the data doesn't exist in the cache or has expired, your application requests the data from your data store. Your data store then returns the data to your application.** Your application next writes the data received from the store to the cache. This way, it can be more quickly retrieved the next time it's requested.

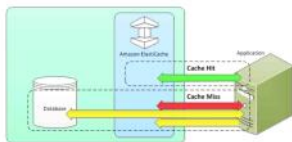
A cache hit occurs when data is in the cache and isn't expired:

1. Your application requests data from the cache.
2. The cache returns the data to the application.

A cache miss occurs when data isn't in the cache or is expired:

1. Your application requests data from the cache.
2. The cache doesn't have the requested data, so returns a null.
3. Your application requests and receives the data from the database.
4. Your application updates the cache with the new data.

The following diagram illustrates both these processes.



via - <https://docs.aws.amazon.com/AmazonElastiCache/latest/mem-ug/Strategies.html>

Write-Through

The write-through strategy adds data or updates data in the cache whenever data is written to the database.

Advantages and Disadvantages of Write-Through

The advantages of write-through are as follows:

- Data in the cache is never stale.
Because the data in the cache is updated every time it's written to the database, the data in the cache is always current.
- Write penalty vs. read penalty.
Every write involves two trips:
 1. A write to the cache
 2. A write to the databaseWhich adds latency to the process. That said, end users are generally more tolerant of latency when updating data than when retrieving data. There is an inherent sense that updates are more work and thus take longer.

The disadvantages of write-through are as follows:

- Missing data.
If you spin up a new node, whether due to a node failure or scaling out, there is missing data. This data continues to be missing until it's added or updated on the database. You can minimize this by implementing **lazy loading** with write-through.
- Cache churn.
Most data is never read, which is a waste of resources. By **adding a time to live (TTL) value**, you can minimize wasted space.

via - <https://docs.aws.amazon.com/AmazonElastiCache/latest/mem-ug/Strategies.html>

Use a caching strategy to write to the backend first and then invalidate the cache

This option is similar to the write-through strategy wherein the application writes to the backend first and then invalidate the cache. As the cache gets invalidated, the caching engine would then fetch the latest value from the backend, thereby making sure that the product prices and product description stay consistent with the backend.

Incorrect options:

Use a caching strategy to update the cache and the backend at the same time - The cache and the backend cannot be updated at the same time via a single atomic operation as these are two separate systems. Therefore this option is incorrect.

Use a caching strategy to write to the backend first and wait for the cache to expire via TTL - This strategy could work if the TTL is really short. However, for the duration of this TTL, the cache would be out of sync with the backend, hence this option is not correct for the given use-case.

Use a caching strategy to write to the cache directly and sync the backend at a later time - This option is given as a distractor as this strategy is not viable to address the given use-case. The product prices and description on the cache must always stay consistent with the backend. You cannot sync the backend at a later time.

Question 61

The customer feedback functionality for a company's flagship web application is handled via an Amazon API Gateway based REST API that invokes an AWS Lambda function for further processing. Although the performance of the function is satisfactory, the development team has been tasked to optimize the startup time of the Lambda function to further improve the customer experience. How will you optimize the Lambda function for faster initialization?

- Configure provisioned concurrency for the Lambda function to respond immediately to the function's invocations
- Enable API caching in Amazon API Gateway to cache AWS Lambda function response
- Configure an interface VPC endpoint powered by AWS PrivateLink to access the Amazon API Gateway REST API with milliseconds latency
- Configure reserved concurrency to guarantee the maximum number of concurrent instances of the Lambda function

Overall explanation

Correct option:

Configure provisioned concurrency for the Lambda function to respond immediately to the function's invocations

When Lambda allocates an instance of your function, the runtime loads your function's code and runs the initialization code that you define outside of the handler. If your code and dependencies are large, or you create SDK clients during initialization, this process can take some time. When your function has not been used for some time, needs to scale up, or when you update a function, Lambda creates new execution environments. This causes the portion of requests that are served by new instances to have higher latency than the rest, otherwise known as a cold start.

By allocating provisioned concurrency before an increase in invocations, you can ensure that all requests are served by initialized instances with low latency. Lambda functions configured with provisioned concurrency run with consistent start-up latency, making them ideal for building interactive mobile or web backends, latency-sensitive microservices, and synchronously invoked APIs.

Functions with Provisioned Concurrency differ from on-demand functions in some important ways:

22. Initialization code does not need to be optimized. Since this happens long before the invocation, lengthy initialization does not impact the latency of invocations. If you are using runtimes that typically take longer to initialize, like Java, the performance of these can benefit from using Provisioned Concurrency.
23. Initialization code is run more frequently than the total number of invocations. Since Lambda is highly available, for every one unit of Provisioned Concurrency, there are a minimum of two execution environments prepared in separate Availability Zones. This is to ensure that your code is available in the event of a service disruption. As environments are reaped and load balancing occurs, Lambda over-provisions environments to ensure availability. You are not charged for this activity. If your code initializer implements logging, you will see additional log files anytime that this code is run, even though the main handler is not invoked.
24. Provisioned Concurrency cannot be used with the \$LATEST version. This feature can only be used with published versions and aliases of a function. If you see cold starts for functions configured to use Provisioned Concurrency, you may be invoking the \$LATEST version, instead of the version or alias with Provisioned Concurrency configured.

Reducing cold starts with Provisioned Concurrency:

Reducing cold starts with Provisioned Concurrency

If you need predictable function start times for your workload, **Provisioned Concurrency** is the recommended solution to ensure the lowest possible latency. This feature keeps your functions initialized and warm, ready to respond in double-digit milliseconds at the scale you provision. Unlike on-demand Lambda functions, this means that all setup activities happen ahead of invocation, including running the initialization code.



For example, a function with a Provisioned Concurrency of 6 has 6 execution environments prepared ahead of when the invocations occur. In the time between initialization and invocation, the execution environment is prepared and ready.



via - <https://aws.amazon.com/blogs/compute/operating-lambda-performance-optimization-part-1/>

Incorrect options:

Configure reserved concurrency to guarantee the maximum number of concurrent instances of the Lambda function - Reserved concurrency guarantees the maximum number of concurrent instances for the function. When a function has reserved concurrency, no other function can use that concurrency. There is no charge for configuring reserved concurrency for a function. Whereas, provisioned concurrency initializes a requested number of execution environments so that they are prepared to respond immediately to your function's invocations.

Configure an interface VPC endpoint powered by AWS PrivateLink to access the Amazon API Gateway REST API with milliseconds latency - An interface VPC endpoint can be used to connect your VPC resources to the AWS Lambda function without crossing the public internet. VPC endpoint is irrelevant to the current discussion.

Enable API caching in Amazon API Gateway to cache AWS Lambda function response - With caching, you can reduce the number of calls made to your AWS Lambda function and also improve the latency of requests to your API. Caching is best-effort and applications making frequent API calls to retrieve static data can benefit from a caching layer. Caching does not reduce the initialization time Lambda takes and hence is not an optimal solution for this use case.

Question 62

A multi-national company maintains separate AWS accounts for different verticals in their organization. The project manager of a team wants to migrate the Elastic Beanstalk environment from Team A's AWS account into Team B's AWS account. As a Developer, you have been roped in to help him in this process.

Which of the following will you suggest?

- Create an export configuration from the Elastic Beanstalk console from Team A's account. This configuration has to be shared with the IAM Role of Team B's account. The import option of Team B's account will show the saved configuration, that can be used to create a new Beanstalk application
- Create a saved configuration in Team A's account and download it to your local machine. Make the account-specific parameter changes and upload to the S3 bucket in Team B's account. From Elastic Beanstalk console, create an application from 'Saved Configurations'
- It is not possible to migrate Elastic Beanstalk environment from one AWS account to the other
- Create a saved configuration in Team A's account and configure it to Export. Now, log into Team B's account and choose the Import option. Here, you need to specify the name of the saved configuration and allow the system to create the new application. This takes a little time based on the Regions the two accounts belong to

Overall explanation

Correct option:

Create a saved configuration in Team A's account and download it to your local machine. Make the account-specific parameter changes and upload to the S3 bucket in Team B's account. From Elastic Beanstalk console, create an application from 'Saved Configurations'

You must use saved configurations to migrate an Elastic Beanstalk environment between AWS accounts. You can save your environment's configuration as an object in Amazon Simple Storage Service (Amazon S3) that can be applied to other environments during environment creation, or applied to a running environment. Saved configurations are YAML formatted templates that define an environment's platform version, tier, configuration option settings, and tags.

Download the saved configuration to your local machine. Change your account-specific parameters in the downloaded configuration file, and then save the changes. For example, change the key pair name, subnet ID, or application name (such as application-b-name). Upload the saved configuration from your local machine to an S3 bucket in Team B's account. From this account, create a new Beanstalk application by choosing 'Saved Configurations' from the navigation panel.

Incorrect options:

Create a saved configuration in Team A's account and configure it to Export. Now, log into Team B's account and choose the Import option. Here, you need to specify the name of the saved configuration and allow the system to create the new application. This takes a little time based on the Regions the two accounts belong to - There is no direct Export and Import option for migrating Elastic Beanstalk configurations.

It is not possible to migrate Elastic Beanstalk environment from one AWS account to the other - This is an incorrect statement.

Create an export configuration from the Elastic Beanstalk console from Team A's account. This configuration has to be shared with the IAM Role of Team B's account. The import option of the Team B's account will show the saved configuration, that can be used to create a new Beanstalk application - This contradicts the explanation provided earlier.

Question 63

The development team at a health-care company is planning to migrate to AWS Cloud from the on-premises data center. The team is evaluating Amazon RDS as the database tier for its flagship application.

Which of the following would you identify as correct for RDS Multi-AZ? (Select two)

- RDS applies OS updates by performing maintenance on the standby, then promoting the standby to primary and finally performing maintenance on the old primary, which becomes the new standby
- Updates to your DB Instance are asynchronously replicated across the Availability Zone to the standby in order to keep both in sync
- Amazon RDS automatically initiates a failover to the standby, in case primary database fails for any reason
- To enhance read scalability, a Multi-AZ standby instance can be used to serve read requests
- For automated backups, I/O activity is suspended on your primary DB since backups are not taken from standby DB

Overall explanation

Correct options:

RDS applies OS updates by performing maintenance on the standby, then promoting the standby to primary, and finally performing maintenance on the old primary, which becomes the

new standby

Running a DB instance as a Multi-AZ deployment can further reduce the impact of a maintenance event because Amazon RDS applies operating system updates by following these steps:

Perform maintenance on the standby.

Promote the standby to primary.

Perform maintenance on the old primary, which becomes the new standby.

When you modify the database engine for your DB instance in a Multi-AZ deployment, then Amazon RDS upgrades both the primary and secondary DB instances at the same time. In this case, the database engine for the entire Multi-AZ deployment is shut down during the upgrade.

Amazon RDS automatically initiates a failover to the standby, in case the primary database fails for any reason - You also benefit from enhanced database availability when running your DB instance as a Multi-AZ deployment. If an Availability Zone failure or DB instance failure occurs, your availability impact is limited to the time automatic failover takes to complete.

Another implied benefit of running your DB instance as a Multi-AZ deployment is that DB instance failover is automatic and requires no administration. In an Amazon RDS context, this means you are not required to monitor DB instance events and initiate manual DB instance recovery in the event of an Availability Zone failure or DB instance failure.

Incorrect options:

For automated backups, I/O activity is suspended on your primary DB since backups are not taken from standby DB - The availability benefits of Multi-AZ also extend to planned maintenance. For example, with automated backups, I/O activity is no longer suspended on your primary during your preferred backup window, since backups are taken from the standby.

To enhance read scalability, a Multi-AZ standby instance can be used to serve read requests - A Multi-AZ standby cannot serve read requests. Multi-AZ deployments are designed to provide enhanced database availability and durability, rather than read scaling benefits. As such, the feature uses synchronous replication between primary and standby. AWS implementation makes sure the primary and the standby are constantly in sync, but precludes using the standby for read or write operations.

Updates to your DB Instance are asynchronously replicated across the Availability Zone to the standby in order to keep both in sync - When you create your DB instance to run as a Multi-AZ deployment, Amazon RDS automatically provisions and maintains a synchronous "standby" replica in a different Availability Zone. Updates to your DB Instance are synchronously replicated across the Availability Zone to the standby in order to keep both in sync and protect your latest database updates against DB instance failure.

Question 64

A developer at a university is encrypting a large XML payload transferred over the network using AWS KMS and wants to test the application before going to production.

What is the maximum data size supported by AWS KMS?

Your answer is correct

- 4KB
- 10MB
- 16KB
- 1MB

Overall explanation

Correct option:

4 KB

You can encrypt up to 4 kilobytes (4096 bytes) of arbitrary data such as an RSA key, a database password, or other sensitive information.

While AWS KMS does support sending data up to 4 KB to be encrypted directly, envelope encryption can offer significant performance benefits. When you encrypt data directly with AWS KMS it must be transferred over the network. Envelope encryption reduces the network load since only the request and delivery of the much smaller data key go over the network. The data key is used locally in your application or encrypting AWS service, avoiding the need to send the entire block of data to AWS KMS and suffer network latency.

Incorrect options:60

1MB - For anything over 4 KB, you may want to look at envelope encryption

10MB - For anything over 4 KB, you may want to look at envelope encryption

16KB - For anything over 4 KB, you may want to look at envelope encryption

Question 65

As an AWS Certified Developer Associate, you are writing a CloudFormation template in YAML. The template consists of an EC2 instance creation and one RDS resource. Once your resources are created you would like to output the connection endpoint for the RDS database.

Which intrinsic function returns the value needed?

- !Ref
- !GetAtt
- !Sub
- !FindInMap

Overall explanation

Correct option:

AWS CloudFormation provides several built-in functions that help you manage your stacks. Intrinsic functions are used in templates to assign values to properties that are not available until runtime.

!GetAtt - The Fn::GetAtt intrinsic function returns the value of an attribute from a resource in the template. This example snippet returns a string containing the DNS name of the load balancer with the logical name myELB - YML : !GetAtt myELB.DNSName JSON : "Fn::GetAtt" : ["myELB" , "DNSName"]

Incorrect options:

!Sub - The intrinsic function Fn::Sub substitutes variables in an input string with values that you specify. In your templates, you can use this function to construct commands or outputs that include values that aren't available until you create or update a stack.

!Ref - The intrinsic function Ref returns the value of the specified parameter or resource.

!FindInMap - The intrinsic function Fn::FindInMap returns the value corresponding to keys in a two-level map that is declared in the Mappings section. For example, you can use this in the Mappings section that contains a single map, RegionMap, that associates AMLs with AWS regions.

Question 66

You are a developer working at a cloud company that embraces serverless. You have performed your initial deployment and would like to work towards adding API Gateway stages and associate them with existing deployments. Your stages will include prod, test, and dev and will need to match a Lambda function variant that can be updated over time.

Which of the following features must you add to achieve this? (select two)

- Lambda X-Ray integration
- Lambda Aliases
- Stage Variables
- Mapping Templates
- Lambda Versions

Overall explanation

Correct options:

Stage Variables

Stage variables are name-value pairs that you can define as configuration attributes associated with a deployment stage of an API. They act like environment variables and can be used in your API setup and mapping templates. With deployment stages in API Gateway, you can manage multiple release stages for each API, such as alpha, beta, and production. Using stage variables you can configure an API deployment stage to interact with different backend endpoints.

For example, your API can pass a GET request as an HTTP proxy to the backend web host (for example, <http://example.com>). In this case, the backend web host is configured in a stage variable so that when developers call your production endpoint, API Gateway calls example.com. When you call your beta endpoint, API Gateway uses the value configured in the stage variable for the beta stage and calls a different web host (for example, beta.example.com).

Lambda Aliases

A Lambda alias is like a pointer to a specific Lambda function version. Users can access the function version using the alias ARN.

Lambda Aliases allow you to create a "mutable" Lambda version that points to whatever version you want in the backend. This allows you to have a "dev", "test", prod" Lambda alias that can remain stable over time.

Incorrect options:

Lambda Versions - Versions are immutable and cannot be updated over time. So this option is not correct.

Lambda X-Ray integration - This is good for tracing and debugging requests so it can be looked at as a good option for troubleshooting issues in the future. This is not the right fit for the given use-case.

Mapping Templates - Mapping template overrides provides you with the flexibility to perform many-to-one parameter mappings; override parameters after standard API Gateway mappings have been applied; conditionally map parameters based on body content or other parameter values; programmatically create new parameters on the fly, and override status codes returned by your integration endpoint. This is not the right fit for the given use-case.

Question 67

A developer hosts a static website in Amazon S3 behind an Amazon CloudFront distribution, using a custom domain that points to the CloudFront URL. The site is deployed through a CI/CD pipeline where the source stage is followed by a build stage that runs an AWS CodeBuild project defined by a buildspec.yml file. The buildspec builds the code and uploads the static files to the S3 bucket, and the updated content is visible both in the bucket and through the S3 website endpoint. However, the CloudFront domain continues to serve stale content.

What do you recommend to fix this issue?

- Enable S3 bucket versioning and keep uploading files under the same keys so CloudFront eventually sees the latest version
- Invoke an Amazon CloudFront invalidation for the changed paths

- Add a post-build step to resync the S3 bucket so the latest artifacts overwrite prior objects
- Modify the S3 bucket's cross-origin resource sharing (CORS) configuration to allow all origins and redeploy the files

Overall explanation

Correct option:

Invoke an Amazon CloudFront invalidation for the changed paths

Invoking an Amazon CloudFront invalidation for the changed paths tells CloudFront to evict those specific objects from its edge caches so the very next viewer request is fetched fresh from your origin (S3) instead of serving a stale copy. You can target just the files you updated or use wildcards (e.g., /images/, /app/.js) to invalidate a whole folder or a set of files sharing a pattern, rather than clearing the entire distribution. This approach forces updates to appear right away, independent of the objects' cache TTLs, and is typically paired with more selective path lists to keep costs low. CloudFront gives you the first 1,000 invalidation paths per month free and then charges per additional path.

Incorrect options:

Modify the S3 bucket's cross-origin resource sharing (CORS) configuration to allow all origins and redeploy the files - CORS settings control how browsers handle requests for resources from different domains (e.g., when JavaScript on one site fetches data from another). While CORS rules are essential for APIs or assets accessed across domains, they do not affect how CloudFront caches and serves static website files. Updating CORS cannot force CloudFront to fetch the latest version of files; stale content will still be served until the cache is invalidated or expires.

Add a post-build step to resync the S3 bucket so the latest artifacts overwrite prior objects - Resyncing ensures the bucket has the newest files, but CloudFront edge caches can continue serving the previously cached objects until their TTLs expire or you explicitly purge them. Simply writing new objects to S3 doesn't invalidate cached copies at CloudFront.

Enable S3 bucket versioning and keep uploading files under the same keys so CloudFront eventually sees the latest version - S3 object versioning tracks multiple versions under the same key, but CloudFront caches by object key (URL) and query string, not by S3 version ID. If you don't change the filename (key) or invalidate the cache, CloudFront will continue serving the cached object until it expires.

Question 68

Your application sends messages to an Amazon Simple Queue Service (SQS) queue frequently, which are then polled by another application that specifies which message to retrieve.

Which of the following options describe the maximum number of messages that can be retrieved at one time?

- 5
- 100
- 10
- 20

Overall explanation

Correct option:

10

After you send messages to a queue, you can receive and delete them. When you request messages from a queue, you can't specify which messages to retrieve. Instead, you specify the maximum number of messages (up to 10) that you want to retrieve.

Incorrect options:

5

20

100

Question 69

A developer has just integrated an AWS Lambda function to an Amazon API Gateway API. The integration has led to errors that the developer is unable to troubleshoot. The developer has decided to enable CloudWatch logging at the method level for the API Gateway API.

What are the key points of consideration while configuring configuring method-level logging for the API Gateway? (Select two)

- AWS Security Token Service(STS) is used by API Gateway for logging data to CloudWatch logs. Hence, AWS STS has to be enabled for the Region that you're using
- You are charged for accessing method-level and stage-level CloudWatch metrics, but not for API-level metrics
- In access logging, only \$context and \$input variables are supported
- API Gateway API log groups or streams can only be deleted and recreated by redeploying the API

To enable CloudWatch Logs for all or only some of the methods, you must also specify the ARN of an IAM role that enables API Gateway to write information to CloudWatch Logs on behalf of your user.

The IAM role must also contain the following trust relationship statement

Overall explanation

Correct options:

AWS Security Token Service(STS) is used by API Gateway for logging data to CloudWatch logs. Hence, AWS STS has to be enabled for the Region that you're using

API Gateway calls AWS Security Token Service to assume the IAM role, so make sure that AWS STS is enabled for the Region. If you receive an error when setting the IAM role ARN, check your AWS Security Token Service account settings to make sure that AWS STS is enabled in the Region that you're using.

To enable CloudWatch Logs for all or only some of the methods, you must also specify the ARN of an IAM role that enables API Gateway to write information to CloudWatch Logs on behalf of your user. The IAM role must also contain the following trust relationship statement

To enable CloudWatch Logs for all or only some of the methods, you must also specify the ARN of an IAM role that enables API Gateway to write information to CloudWatch Logs on behalf of your user.

To do so, choose Settings from the APIs main navigation pane. Then enter the ARN of an IAM role in the CloudWatch log role ARN text field. The IAM role must also contain the trust relationship statement.

Policy of AmazonAPIGatewayPushToCloudWatchLogs for IAM role:



via - <https://docs.aws.amazon.com/apigateway/latest/developerguide/stages.html#how-to-stage-settings-console>

Incorrect options:

In access logging, only \$context and \$input variables are supported - This statement is incorrect. In access logging, you, as an API developer, want to log who has accessed your API and how the caller accessed the API. You can create your own log group or choose an existing log group that could be managed by API Gateway. To specify the access details, you select \$context variables and choose a log group as the destination. Only \$context variables are supported (not \$input, and so on).

You are charged for accessing method-level and stage-level CloudWatch metrics, but not for API-level metrics - This statement is incorrect. Your account is charged for accessing method-level CloudWatch metrics, but not the API-level or stage-level metrics.

API Gateway API log groups or streams can only be deleted and recreated by redeploying the API - API Gateway API log groups or streams can be deleted from the CloudWatch console. But, it is not recommended.

Do not manually delete API Gateway API log groups or streams; let API Gateway manage these resources. Manually deleting log groups or streams may cause API requests and responses not to be logged. If that happens, you can delete the entire log group for the API and redeploy the API. This is because API Gateway creates log groups or log streams for an API stage at the time when it is deployed.